

Recursive Projection-Aggregation Decoding of Reed-Muller Codes

Min Ye¹ and Emmanuel Abbe

Abstract—We propose a new class of efficient decoding algorithms for Reed-Muller (RM) codes over binary-input memoryless channels. The algorithms are based on projecting the code on its cosets, recursively decoding the projected codes (which are lower-order RM codes), and aggregating the reconstructions (e.g., using majority votes). We further provide extensions of the algorithms using list-decoding. We run our algorithm for AWGN channels and Binary Symmetric Channels at the short code length (≤ 1024) regime for a wide range of code rates. Simulation results show that in both low code rate and high code rate regimes, the new algorithm outperforms the widely used decoder for polar codes (SCL+CRC) with the same parameters. The performance of the new algorithm for RM codes in those regimes is in fact close to that of the maximal likelihood decoder. Finally, the new decoder naturally allows for parallel implementations.

Index Terms—Reed-Muller codes, polar codes, RPA decoding, AWGN channels, binary symmetric channels.

I. INTRODUCTION

REED-MULLER (RM) codes are among the oldest families of error-correcting codes [2]. The recent breakthrough of polar codes [3] has brought the attention back to RM codes, due to the closeness of the two codes. RM codes have in particular the advantage of having a simple and universal code construction, and promising performances were demonstrated in several works [4], [5], with a scaling law conjectured to be comparable of that of random codes.

RM codes do not possess yet the generic analytical framework of polar codes (i.e., polarization theory). It was recently shown that RM codes achieve capacity on the Binary Erasure Channel (BEC) at constant rate [6], as well as for extremal rates for BEC and Binary Symmetric Channels (BSC) [7], but obtaining such results for a broader class of communication channels and rates remains open. Recent progress was made

on these questions with a polarization approach to RM codes shown in [8]. See also [9] for a recent survey on RM codes.

Various decoding algorithms have been proposed for RM codes, starting with Reed algorithm [2], [10], and four important more recent line of works including automorphism group based decoding [11]–[13], recursive list-decoding [14]–[16], a new Berlekamp-Welch type of algorithm [17], [18], and a new algorithm utilizing minimum-weight parity checks [19]. In particular, [11], [14]–[18] give fairly powerful theoretical guarantees for efficient decoding of RM codes in specific regimes. However, there is not a thorough comparison between the performance of RM codes under these decoders and the performance of the widely used CRC-aided polar codes under the Successive Cancellation List (SCL) decoders [20].

In this paper, we propose a new class of decoding algorithms for Reed-Muller codes over any binary-input memoryless channels and compare its performance with polar codes. The new algorithms are based on recursive projections and aggregations of cosets decoding, exploiting the self-similarity of RM codes, and are extended with Chase list-decoding algorithms [21]. We run our new algorithms at the short code length (≤ 1024) regime for a wide range of code rates. Simulation results show that the new algorithms improve on the widely used decoding algorithm for polar codes [20] in both low code rate and high code rate regimes. These are the type of regimes where polar codes are planned to enter the 5G standards [22] as well as relevant regimes for applications in the Internet of Things (IoT).

More specifically, we compare our new algorithm for RM codes with the Successive Cancellation List (SCL) decoder for CRC-aided polar codes [20], where we set the CRC size to take optimal values.¹ For AWGN channels, our new algorithm has about 0.5dB gain (more in some cases) over polar codes in various short code length (≤ 1024) and low code rate (≤ 0.5) regimes, and similar improvements are also obtained for BSC channels. Moreover, the performance of our new decoding algorithm is comparable to the best previously known algorithms for RM codes [16].

In the above regimes, the decoding error probability of our new algorithm is in fact shown to be close to that of the Maximal Likelihood decoder on RM codes. Some extensions and variants to potentially further improve the performance are also discussed, as well as possible extensions of the projection-aggregation algorithms to other families of codes.

¹The optimal CRC size depends on the choice of code length and rate.

Manuscript received April 27, 2019; revised November 5, 2019; accepted February 22, 2020. Date of publication March 3, 2020; date of current version July 14, 2020. This article was presented in part at the 2019 IEEE International Symposium on Information Theory. (Corresponding author: Min Ye.)

Min Ye is with the Data Science and Information Technology Research Center, Tsinghua-Berkeley Shenzhen Institute, Shenzhen 518055, China (e-mail: yeemmi@gmail.com).

Emmanuel Abbe is with the School of Computer and Communication Sciences, Mathematics Institute, École Polytechnique Fédérale de Lausanne (EPFL), 1015 Lausanne, Switzerland, and also with the Program in Applied and Computational Mathematics, Department of Electrical Engineering, Princeton University, Princeton, NJ 08544 USA.

Communicated by J. Kliewer, Associate Editor for Coding Techniques.

Color versions of one or more of the figures in this article are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TIT.2020.2977917

In Section II, we give a high level description of the new type of algorithms. In Section III, we present decoding algorithm for BSC channels. In Section IV we generalize the algorithms to decode RM codes over any binary-input channel. Finally, in Section VI we present simulation results. In addition to the previously mentioned improvements over polar codes, we also empirically validate the improved scaling-law of RM codes over polar codes on BSC channels [23].

II. A HIGH-LEVEL DESCRIPTION OF THE NEW ALGORITHMS

We begin with some notation and background on RM codes. In this paper, we use $\oplus$ to denote sums over $\mathbb{F}_2$. Let us consider the polynomial ring $\mathbb{F}_2[Z_1, Z_2, \dots, Z_m]$ of m variables. Since $Z^2 = Z$ in $\mathbb{F}_2$, the following set of 2^m monomials forms a basis of $\mathbb{F}_2[Z_1, Z_2, \dots, Z_m]$:

$$\left\{ \prod_{i \in A} Z_i : A \subseteq [m] \right\}, \text{ where } \prod_{i \in \emptyset} Z_i := 1.$$

Next we associate every subset $A \subseteq [m]$ with a row vector $\mathbf{v}_m(A)$ of length 2^m , whose components are indexed by a binary vector $\mathbf{z} = (z_1, z_2, \dots, z_m) \in \{0, 1\}^m$. The vector $\mathbf{v}_m(A)$ is defined as follows:

$$\mathbf{v}_m(A, \mathbf{z}) = \prod_{i \in A} z_i, \quad (1)$$

where $\mathbf{v}_m(A, \mathbf{z})$ is the component of $\mathbf{v}_m(A)$ indexed by $\mathbf{z}$, i.e., $\mathbf{v}_m(A, \mathbf{z})$ is the evaluation of the monomial $\prod_{i \in A} Z_i$ at $\mathbf{z}$. For $0 \leq r \leq m$, the set of vectors

$$\{\mathbf{v}_m(A) : A \subseteq [m], |A| \leq r\}$$

forms a basis of the r -th order Reed-Muller code $\mathcal{RM}(m, r)$ of length $n := 2^m$ and dimension $\sum_{i=0}^r \binom{m}{i}$.

Definition 1: The r -th order Reed-Muller code $\mathcal{RM}(m, r)$ code is defined as the following set of binary vectors

$$\mathcal{RM}(m, r) := \left\{ \sum_{A \subseteq [m], |A| \leq r} u(A) \mathbf{v}_m(A) : u(A) \in \{0, 1\} \right. \\ \left. \text{for all } A \subseteq [m], |A| \leq r \right\}.$$

In other words, each vector $\mathbf{v}_m(A)$ consists of all the evaluations of the monomial $\prod_{i \in A} Z_i$ at all the points in the vector space $\mathbb{E} := \mathbb{F}_2^m$, and each codeword $c \in \mathcal{RM}(m, r)$ corresponds to an m -variate polynomial with degree at most r . The coordinates of the codeword c are also indexed by the binary vectors $\mathbf{z} \in \mathbb{E}$, and we write $c = (c(\mathbf{z}), \mathbf{z} \in \mathbb{E})$. Let $\mathbb{B}$ be an s -dimensional subspace of $\mathbb{E}$, where $s \leq r$. The quotient space $\mathbb{E}/\mathbb{B}$ consists of all the cosets of $\mathbb{B}$ in $\mathbb{E}$, where every coset T has form $T = \mathbf{z} + \mathbb{B}$ for some $\mathbf{z} \in \mathbb{E}$. For a binary vector $y = (y(\mathbf{z}), \mathbf{z} \in \mathbb{E})$, we define its projection on the cosets of $\mathbb{B}$ as

$$y_{/\mathbb{B}} = \text{Proj}(y, \mathbb{B}) := \left(y_{/\mathbb{B}}(T), T \in \mathbb{E}/\mathbb{B} \right), \quad (2)$$

where $y_{/\mathbb{B}}(T) := \bigoplus_{\mathbf{z} \in T} y(\mathbf{z})$ is the binary vector obtained by summing up all the coordinates of y in each coset $T \in \mathbb{E}/\mathbb{B}$. Here the sum is over $\mathbb{F}_2$ and the dimension of $y_{/\mathbb{B}}$ is $n/|\mathbb{B}|$.

In the next section, we will show that if c is a codeword of $\mathcal{RM}(m, r)$, then $c_{/\mathbb{B}}$ is a codeword of $\mathcal{RM}(m-s, r-s)$,

Algorithm 1 The RPA_RM Decoding Function for BSC

Input: The corrupted codeword $y = (y(\mathbf{z}), \mathbf{z} \in \mathbb{E})$; the parameters of the Reed-Muller code m and r ; the maximal number of iterations $N_{\max}$

Output: The decoded codeword $\hat{c}$

```

1: for  $j = 1, 2, \dots, N_{\max}$  do
2:    $y_{/\mathbb{B}_i} \leftarrow \text{Proj}(y, \mathbb{B}_i)$  for  $i = 1, 2, \dots, 2^m - 1$ 
3:    $\hat{y}_{/\mathbb{B}_i} \leftarrow \text{RPA\_RM}(y_{/\mathbb{B}_i}, m-1, r-1, N_{\max})$  for  $i = 1, 2, \dots, 2^m - 1$ 
4:    $\triangleright$  If  $r = 2$ , then we use the Fast Hadamard Transform to decode the first-order RM code [10]
5:    $\hat{y} \leftarrow \text{Aggregation}(y, \hat{y}_{/\mathbb{B}_1}, \hat{y}_{/\mathbb{B}_2}, \dots, \hat{y}_{/\mathbb{B}_{n-1}})$ 
6:   if  $y = \hat{y}$  then
7:     break  $\triangleright y = \hat{y}$  means that the algorithm already converges to a fixed (stable) point
8:   end if
9:    $y \leftarrow \hat{y}$ 
10: end for
11:  $\hat{c} \leftarrow \hat{y}$ 
12: return  $\hat{c}$ 

```

where s is the dimension of $\mathbb{B}$. Our new decoding algorithm makes use of the case $s = 1$, namely, the one-dimensional subspaces. More precisely, let $y = (y(\mathbf{z}), \mathbf{z} \in \mathbb{E})$ be the output vector of transmitting a codeword of $\mathcal{RM}(m, r)$ over some BSC channel. Our decoding algorithm is defined in a recursive way: For every one-dimensional subspace $\mathbb{B}$, we first obtain the projection $y_{/\mathbb{B}}$, and then we use the decoding algorithm for $\mathcal{RM}(m-1, r-1)$ to decode $y_{/\mathbb{B}}$, where the decoding result is denoted as $\hat{y}_{/\mathbb{B}}$. Since every one-dimensional subspace of $\mathbb{E}$ consists of 0 and a non-zero element, there are $n-1$ such subspaces in total. After the projection and recursive decoding steps, we obtain $n-1$ decoding results $\hat{y}_{/\mathbb{B}_1}, \hat{y}_{/\mathbb{B}_2}, \dots, \hat{y}_{/\mathbb{B}_{n-1}}$. Next we use a majority voting scheme to aggregate these decoding results together with y to obtain a new estimate $\hat{y}$ of the original codeword. Finally we update y as $\hat{y}$, and iterate the whole procedure for up to $N_{\max}$ rounds. Notice that if $y = \hat{y}$ (see line 6), then y is a fixed (stable) point of this algorithm and will remain unchanged for the next iterations. In this case we should exit the for loop on line 1 (see line 6–8). In practice we set the maximal number of iterations $N_{\max} = \lceil m/2 \rceil$ to prevent the program from running into an infinite loop, and typically $\lceil m/2 \rceil$ iterations are enough for the algorithm to converge to a stable y . This high-level description is summarized in Fig. 1 and Algorithm 1. While this description focuses on the decoding algorithm over BSC, a natural extension of this algorithm bases on log-likelihood ratios (LLRs) allows us to decode RM codes over any binary-input memoryless channels, including the AWGN channel; see Section IV for details.

A. List Decoding Procedure [21]

Here we recap (a version of) the list decoding procedure proposed by Chase [21] that can further decrease the decoding error probability. Suppose that we have a unique decoding algorithm `decodeC` for some code $\mathcal{C}$ over some binary-input memoryless channel $W : \{0, 1\} \rightarrow \mathcal{W}$. Without loss of

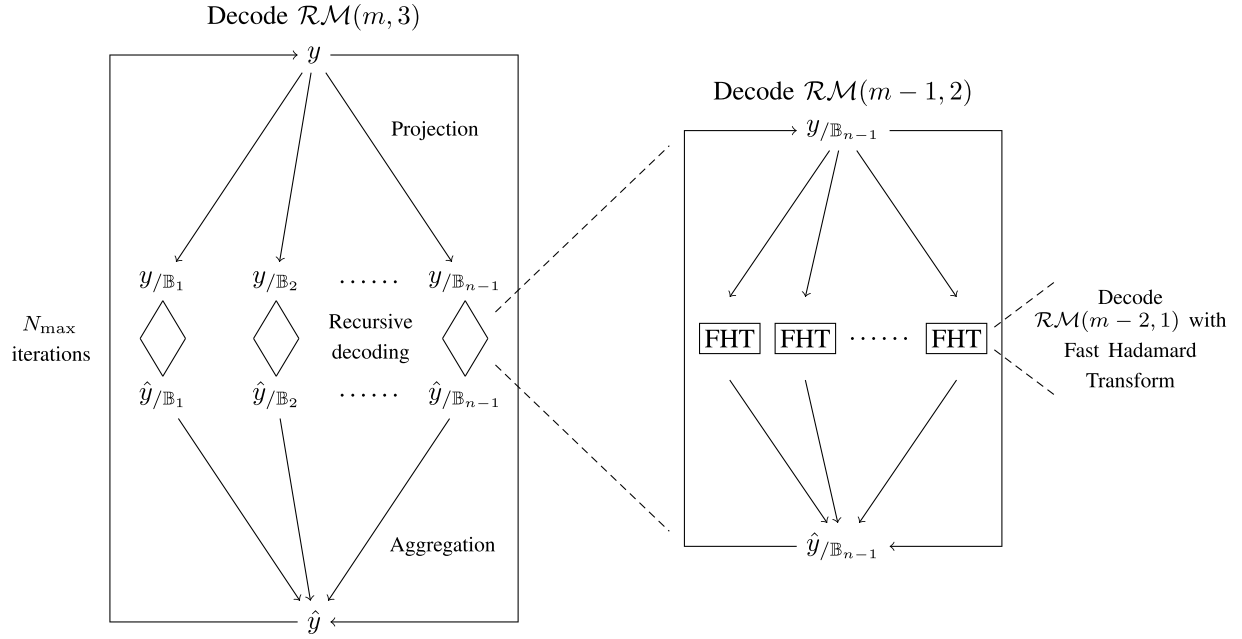


Fig. 1. Recursive Projection-Aggregation decoding algorithm for third order RM codes.

generality, assume that `decodeC` is based on the LLR vector of the channel output, where the LLR of an output symbol $x \in \mathcal{W}$ is defined as

$$\text{LLR}(x) := \ln \left(\frac{W(x|0)}{W(x|1)} \right). \quad (3)$$

Clearly, if $|\text{LLR}(x)|$ is small, then x is a noisy symbol, and if $|\text{LLR}(x)|$ is large, then x is relatively noiseless.

The list decoding procedure works as follows. Suppose that $y = (y_1, y_2, \dots, y_n)$ is the output vector when we send a codeword of $\mathcal{C}$ over the channel $\mathcal{W}$. We first sort $|\text{LLR}(y_i)|, i \in [n]$ from small to large. Without loss of generality, let us assume that $|\text{LLR}(y_1)|, |\text{LLR}(y_2)|, |\text{LLR}(y_3)|$ are the three smallest components in the LLR vector, meaning that y_1, y_2 and y_3 are the three most noisy symbols in the channel outputs (we take three arbitrarily). Next we enumerate all the possible cases of the first three bits of the codeword $c = (c_1, c_2, \dots, c_n)$: The first three bits (c_1, c_2, c_3) can be any vector in $\mathbb{F}_2^3$, so there are 8 cases in total, and for each case we change the value of $\text{LLR}(y_1), \text{LLR}(y_2), \text{LLR}(y_3)$ according to the values of c_1, c_2, c_3 . More precisely, we set $\text{LLR}(y_i) = (-1)^{c_i} L_{\max}$ for $i = 1, 2, 3$, where $L_{\max}$ is some large real number. In practice, we can choose $L_{\max} := \max(|\text{LLR}(y_i)|, i \in [n])$ or $L_{\max} := 2 \max(|\text{LLR}(y_i)|, i \in [n])$. For each of these 8 cases, we use `decodeC` to obtain a decoded codeword, and we denote them as $\hat{c}^{(1)}, \hat{c}^{(2)}, \dots, \hat{c}^{(8)}$. Finally, we calculate the posterior probability of $W^n(y|\hat{c}^{(i)}), 1 \leq i \leq 8$, and choose the largest one as the final decoding result, namely, we perform a maximal likelihood decoding among the 8 candidates in the list.

When we apply this list decoding procedure together with Algorithm 1 to decode RM codes, the decoding error probability is typically close to that of the Maximal Likelihood decoder.

III. DECODING ALGORITHM FOR BSC

We begin with the definition of the quotient code. Then we show that the quotient code of an RM code is also an RM code.

Definition 2: Let $s \leq r \leq m$ be integers, and let $\mathbb{B}$ be an s -dimensional subspace of $\mathbb{E} := \mathbb{F}_2^m$. We define the quotient code

$$\mathcal{Q}(m, r, \mathbb{B}) := \{c_{/\mathbb{B}} : c \in \mathcal{RM}(m, r)\}.$$

Lemma 1: Let $s \leq r \leq m$ be integers, and let $\mathbb{B}$ be an s -dimensional subspace of $\mathbb{E} := \mathbb{F}_2^m$. The code $\mathcal{Q}(m, r, \mathbb{B})$ is the Reed-Muller code $\mathcal{RM}(m-s, r-s)$.

This lemma is an immediate corollary of Theorem 12 in [10, Chapter 13]. For the sake of completeness, we give a proof of this lemma in Appendix A.

Note that Reed's algorithm [2] relies on the special case of $s = r$ in Lemma 1, and our new decoding algorithm makes use of the case $s = 1$ in Lemma 1 (in addition to using all subspaces and adding an iterative process). The `RPA_RM` decoding function is already presented in the previous section. Here we fill in the only missing component, namely the Aggregation function; see Algorithm 2 below. Both $y_{/\mathbb{B}_i} = (y_{/\mathbb{B}_i}(T), T \in \mathbb{E}/\mathbb{B})$ and $\hat{y}_{/\mathbb{B}_i} = (\hat{y}_{/\mathbb{B}_i}(T), T \in \mathbb{E}/\mathbb{B})$ are indexed by the cosets $T \in \mathbb{E}/\mathbb{B}$, and we use $[z + \mathbb{B}]$ to denote the coset containing z (see line 3).

From line 3, we can see that the maximal possible value of $\text{changevote}(z)$ for each $z \in \mathbb{E}$ is $n-1$. Therefore the condition $\text{changevote}(z) > \frac{n-1}{2}$ on line 4 can indeed be viewed as a majority vote. As discussed in Section III-A, this algorithm can be viewed as one step of the power iteration method to find the eigenvector of a matrix built from the quotient code decoding.

In Algorithms 1–2, we write the pseudo codes in a mathematical fashion for the ease of understanding. In Appendix C,

Algorithm 2 The Aggregation Function for BSC**Input:** $y, \hat{y}_{/\mathbb{B}_1}, \hat{y}_{/\mathbb{B}_2}, \dots, \hat{y}_{/\mathbb{B}_{n-1}}$ **Output:** y

- 1: Initialize $(\text{changevote}(\mathbf{z}), \mathbf{z} \in \{0, 1\}^m)$ as an all-zero vector indexed by $\mathbf{z} \in \{0, 1\}^m$
- 2: $n \leftarrow 2^m$
- 3: $\text{changevote}(\mathbf{z}) \leftarrow \sum_{i=1}^{n-1} \mathbb{1}[y_{/\mathbb{B}_i}([\mathbf{z} + \mathbb{B}_i]) \neq \hat{y}_{/\mathbb{B}_i}([\mathbf{z} + \mathbb{B}_i])]$ for each $\mathbf{z} \in \{0, 1\}^m$
- 4: $y(\mathbf{z}) \leftarrow y(\mathbf{z}) \oplus \mathbb{1}[\text{changevote}(\mathbf{z}) > \frac{n-1}{2}]$ for each $\mathbf{z} \in \{0, 1\}^m$ $\triangleright$ Here addition is over $\mathbb{F}_2$
- 5: **return** y

we present another version of the RPA_RM function in a program language fashion; see Algorithm 8.

Proposition 1: The complexity of Algorithm 1 is $O(n^r \log n)$ in sequential implementation and $O(n^2)$ in parallel implementation with $O(n^r)$ processors.

In Section VI-C, we further discuss options to reduce the computation time by using fewer subspaces in the projection step.

Proof: We prove by the induction on the order of the RM code. To establish the base case, observe that the complexity of decoding first-order RM codes using Fast Hadamard Transform (FHT) [10], [24] is $O(n \log n)$. Now we assume the proposition holds for decoding $(r-1)$ -th order RM codes and prove the inductive step. Clearly, the complexity of Algorithm 1 is determined by the complexity of the recursive decoding step on line 3. By induction hypothesis, the complexity of decoding each $y_{/\mathbb{B}_i}$ is $O(n^{r-1} \log n)$. Since there are $n-1$ one-dimensional subspaces $\mathbb{B}_1, \mathbb{B}_2, \dots, \mathbb{B}_{n-1}$, the complexity of Algorithm 1 is indeed $O(n^r \log n)$. $\square$

In the next proposition, we show that whether Algorithm 1 outputs the correct codeword or not is independent of the transmitted codeword and only depends on the error pattern imposed by the BSC channel.

Proposition 2: Let $c \in \mathcal{RM}(m, r)$ be a codeword of the RM code. Let $e = (e(\mathbf{z}), \mathbf{z} \in \mathbb{E})$ be the error vector imposed on c by the BSC channel, and the output vector of the BSC channel is $y = c + e$. Denote the decoding result as $\hat{c} = \text{RPA_RM}(y, m, r, N_{\max})$. Then the indicator function of decoding error $\mathbb{1}[\hat{c} \neq c]$ is independent of the choice of c and only depends on the error vector e .

Notice that we use maximal likelihood decoder for first-order RM code, and the proposition can be proved by induction on the order of the RM code.² This proposition is useful for simulations because we can simply transmit the all-zero codeword over the BSC channel to measure the decoding error probability.

A. Spectral Interpretations of Algorithm 2

Algorithm 2 can be viewed as a one-step power iteration of a spectral algorithm. More precisely, observe that $\hat{y}_{/\mathbb{B}_1}, \hat{y}_{/\mathbb{B}_2}, \dots, \hat{y}_{/\mathbb{B}_{n-1}}$ contain the estimates of $c(\mathbf{z}) \oplus c(\mathbf{z}')$ for

all $\mathbf{z} \neq \mathbf{z}'$, where $c = (c(\mathbf{z}), \mathbf{z} \in \mathbb{E})$ is the transmitted (true) codeword. {We denote the estimate of $c(\mathbf{z}) \oplus c(\mathbf{z}')$ as $\hat{y}_{\mathbf{z}, \mathbf{z}'}$. Suppose for the moment that we want to find a vector $\hat{y} = (\hat{y}(\mathbf{z}), \mathbf{z} \in \mathbb{E}) \in \{0, 1\}^n$ to agree with as many estimates of these sums as possible, i.e., we want to find a vector $\hat{y}$ to maximize

$$|\{(\mathbf{z}, \mathbf{z}') : \mathbf{z} \neq \mathbf{z}', \hat{y}(\mathbf{z}) \oplus \hat{y}(\mathbf{z}') = \hat{y}_{\mathbf{z}, \mathbf{z}'}\}|.$$

Notice that

$$\begin{aligned} & |\{(\mathbf{z}, \mathbf{z}') : \mathbf{z} \neq \mathbf{z}', \hat{y}(\mathbf{z}) \oplus \hat{y}(\mathbf{z}') = \hat{y}_{\mathbf{z}, \mathbf{z}'}\}| \\ & + |\{(\mathbf{z}, \mathbf{z}') : \mathbf{z} \neq \mathbf{z}', \hat{y}(\mathbf{z}) \oplus \hat{y}(\mathbf{z}') \neq \hat{y}_{\mathbf{z}, \mathbf{z}'}\}| = n(n-1). \end{aligned}$$

Therefore,

$$\begin{aligned} & \sum_{\mathbf{z} \neq \mathbf{z}'} (-1)^{\hat{y}(\mathbf{z}) + \hat{y}(\mathbf{z}') + \hat{y}_{\mathbf{z}, \mathbf{z}'}} \\ & = 2 |\{(\mathbf{z}, \mathbf{z}') : \mathbf{z} \neq \mathbf{z}', \hat{y}(\mathbf{z}) \oplus \hat{y}(\mathbf{z}') = \hat{y}_{\mathbf{z}, \mathbf{z}'}\}| - n(n-1). \end{aligned}$$

Thus our task is equivalent to find

$$\arg\max_{\hat{y} \in \{0, 1\}^n} \sum_{\mathbf{z} \neq \mathbf{z}'} (-1)^{\hat{y}(\mathbf{z}) + \hat{y}(\mathbf{z}') + \hat{y}_{\mathbf{z}, \mathbf{z}'}}. \quad (4)$$

Given a vector $\hat{y} \in \{0, 1\}^n$, we define another vector $\hat{u} \in \{-1, 1\}^n$ by setting $\hat{u}(\mathbf{z}) := (-1)^{\hat{y}(\mathbf{z})}$ for all $\mathbf{z} \in \mathbb{E}$. In order to find the maximizing vector $\hat{y}$ in (4), it suffices to find

$$\arg\max_{\hat{u} \in \{-1, 1\}^n} \sum_{\mathbf{z} \neq \mathbf{z}'} (-1)^{\hat{y}_{\mathbf{z}, \mathbf{z}'}} \hat{u}(\mathbf{z}) \hat{u}(\mathbf{z}'). \quad (5)$$

Now we build an $n \times n$ matrix A from $\{\hat{y}_{\mathbf{z}, \mathbf{z}'} : \mathbf{z}, \mathbf{z}' \in \mathbb{E}, \mathbf{z} \neq \mathbf{z}'\}$ as follows: The rows and columns of A are indexed by $\mathbf{z} \in \mathbb{E}$, and we set the entry

$$A_{\mathbf{z}, \mathbf{z}'} := \begin{cases} (-1)^{\hat{y}_{\mathbf{z}, \mathbf{z}'}} & \text{if } \mathbf{z} \neq \mathbf{z}' \\ 0 & \text{if } \mathbf{z} = \mathbf{z}' \end{cases},$$

i.e., for $\mathbf{z} \neq \mathbf{z}'$ we set $A_{\mathbf{z}, \mathbf{z}'} = 1$ if $\hat{y}_{\mathbf{z}, \mathbf{z}'} = 0$, and $A_{\mathbf{z}, \mathbf{z}'} = -1$ if $\hat{y}_{\mathbf{z}, \mathbf{z}'} = 1$. Under this definition, the optimization problem (5) becomes

$$\begin{aligned} & \arg\max_{\hat{u} \in \{-1, 1\}^n} \sum_{\mathbf{z} \neq \mathbf{z}'} A_{\mathbf{z}, \mathbf{z}'} \hat{u}(\mathbf{z}) \hat{u}(\mathbf{z}') \\ & = \arg\max_{\hat{u} \in \{-1, 1\}^n} \hat{u}^T A \hat{u}. \end{aligned} \quad (6)$$

It is well known that this combinatorial optimization problem is NP-hard. In practice, people usually use the following spectral relaxation to obtain approximate solution:

$$\arg\max_{\hat{u} \in \mathbb{R}^n, \|\hat{u}\|^2 = n} \hat{u}^T A \hat{u}.$$

It is well known that the solution to this relaxed optimization problem is the eigenvector corresponding to the largest eigenvalue of A . One way to find this eigenvector is to use the power iteration method: pick some vector v (e.g., at random), then $A^t v$ converges to this eigenvector when t is large enough.³ After rescaling $A^t v$ to make $\|A^t v\|^2 = n$, we obtain the maximizing vector $\hat{u} = A^t v$ in the relaxed optimization problem. In order to obtain the solution to the original optimization problem in (6), we only need to look at the sign of each coordinate of $\hat{u}$: If $\hat{u}(\mathbf{z}) > 0$, then we set

³Assume the largest eigenvalue has largest magnitude.

²See the proof of Proposition 4 for a rigorous argument. The ideas of the proofs of these two propositions are exactly the same.

$\hat{u}(z) = 1$, and if $\tilde{u}(z) < 0$, then we set $\hat{u}(z) = -1$. In this way, we obtain the vector $\hat{u}$ that serves as our approximate solution to (6). To summarize, our approximate solution to (6) is $\hat{u} = \text{sign}(A^t v)$, where v is some random vector and t is some large enough integer.

Let us denote the output vector of Algorithm 2 as $\bar{y}$, and we define another vector $\bar{u}$ as $\bar{u}(z) = (-1)^{\bar{y}(z)}$ for all $z \in \mathbb{E}$. For the original received vector y , we also define a vector u as $u(z) = (-1)^{y(z)}$ for all $z \in \mathbb{E}$. The main observation in this subsection is that

$$\bar{u} = \text{sign}(A\bar{y}), \quad (7)$$

i.e., the output of Algorithm 2 is in fact the same as a one-step power iteration of the spectral algorithm with the original received vector u playing the role of vector v above. It is also easy to see why (7) holds: According to (7), $\bar{u}(z) = 1$ if $\sum_{z' \neq z} (-1)^{\hat{y}_{z,z'} \oplus y(z')} > 0$ and $\bar{u}(z) = -1$ otherwise. This is equivalent to saying that $\bar{y}(z) = 0$ if $|\{z' : z' \neq z, \hat{y}_{z,z'} \oplus y(z') = 0\}| > \frac{n-1}{2}$ and $\bar{u}(z) = 1$ otherwise. Clearly, the vector $\bar{y}$ given by this rule is exactly the same as the output vector of Algorithm 2.

We tried to use the power-iteration method in the Aggregation function for more than one step. However, the performance does not improve over the current version of Aggregation function based on majority vote. This is because in the spectral method above we tried our best to agree with $\hat{y}_{/\mathbb{B}_1}, \hat{y}_{/\mathbb{B}_2}, \dots, \hat{y}_{/\mathbb{B}_{n-1}}$, ignoring the original channel output y , and many of these are very noisy measurements.

IV. DECODING ALGORITHM FOR GENERAL BINARY-INPUT MEMORYLESS CHANNELS

The decoding algorithm in the previous section only works for the BSC. In this section, we will present a natural extension of Algorithm 1 that works for any binary-input memoryless channels, and this new algorithm is based on LLRs (see (3)). Similarly to Algorithm 1, this new algorithm is also defined recursively, i.e., we first assume that we know how to decode $(r-1)$ -th order Reed-Muller code, and then we use it to decode the r -th order Reed-Muller code. To begin with, note that the soft-decision FHT decoder [25] allows us to decode the first order RM code efficiently for general binary-input channels. The soft-decision FHT decoder is based on LLR, and the complexity is also $O(n \log n)$, the same as the hard-decision FHT decoder.

For completeness, we recap the FHT decoder in [25] for first order RM codes. We still use $c = (c(z), z \in \mathbb{E})$ to denote the transmitted (true) codeword and $y = (y(z), z \in \mathbb{E})$ to denote the corresponding channel output. Given the output vector y , the ML decoder for first order RM codes aims to find $c \in \mathcal{RM}(m, 1)$ to maximize $\prod_{z \in \mathbb{E}} W(y(z)|c(z))$. This is equivalent to maximizing the following quantity:

$$\prod_{z \in \mathbb{E}} \frac{W(y(z)|c(z))}{\sqrt{W(y(z)|0)W(y(z)|1)}},$$

which is further equivalent to maximizing

$$\sum_{z \in \mathbb{E}} \ln \left(\frac{W(y(z)|c(z))}{\sqrt{W(y(z)|0)W(y(z)|1)}} \right). \quad (8)$$

Notice that the codeword c is a binary vector. Therefore,

$$\begin{aligned} & \ln \left(\frac{W(y(z)|c(z))}{\sqrt{W(y(z)|0)W(y(z)|1)}} \right) \\ &= \begin{cases} \frac{1}{2} \text{LLR}(y(z)) & \text{if } c(z) = 0 \\ -\frac{1}{2} \text{LLR}(y(z)) & \text{if } c(z) = 1 \end{cases}. \end{aligned}$$

From now on we will use the shorthand notation

$$L(z) := \text{LLR}(y(z)),$$

and the formula in (8) can be written as

$$\frac{1}{2} \sum_{z \in \mathbb{E}} \left((-1)^{c(z)} L(z) \right), \quad (9)$$

so we want to find $c \in \mathcal{RM}(m, 1)$ to maximize this quantity.

By definition, every $c \in \mathcal{RM}(m, 1)$ corresponds to a polynomial in $\mathbb{F}_2[Z_1, Z_2, \dots, Z_m]$ of degree one, so we can write every codeword c as a polynomial $u_0 + \sum_{i=1}^m u_i Z_i$. In this way, we have $c(z) = u_0 + \sum_{i=1}^m u_i z_i$, where $z_1, z_2, \dots, z_m$ are the coordinates of the vector z . Now our task is to find $u_0, u_1, u_2, \dots, u_m \in \mathbb{F}_2$ to maximize

$$\begin{aligned} & \sum_{z \in \mathbb{E}} \left((-1)^{u_0 + \sum_{i=1}^m u_i z_i} L(z) \right) \\ &= (-1)^{u_0} \sum_{z \in \mathbb{E}} \left((-1)^{\sum_{i=1}^m u_i z_i} L(z) \right). \end{aligned} \quad (10)$$

For a binary vector $u = (u_1, u_2, \dots, u_m) \in \mathbb{E}$, we define

$$\hat{L}(u) := \sum_{z \in \mathbb{E}} \left((-1)^{\sum_{i=1}^m u_i z_i} L(z) \right).$$

Clearly, to find the maximizer of (10), we only need to calculate $\hat{L}(u)$ for all $u \in \mathbb{E}$, but the vector $(\hat{L}(u), u \in \mathbb{E})$ is exactly the Hadamard Transform of the vector $(L(z), z \in \mathbb{E})$, so it can be calculated using the Fast Hadamard Transform with complexity $O(n \log n)$. Once we know the values of $(\hat{L}(u), u \in \mathbb{E})$, we can find $u^* = (u_1^*, u_2^*, \dots, u_m^*) \in \mathbb{E}$ that maximizes $|\hat{L}(u)|$. If $\hat{L}(u^*) > 0$, then the decoder outputs the codeword corresponding to $u_0^* = 0, u_1^*, u_2^*, \dots, u_m^*$. Otherwise, the decoder outputs the codeword corresponding to $u_0^* = 1, u_1^*, u_2^*, \dots, u_m^*$. This completes the description of how to decode the first order RM codes for general channels.

The next problem is how to extend (2) in the general setting. The purpose of (2) is mapping two output symbols $(y(z), z \in T)$ whose indices are in the same coset $T \in \mathbb{E}/\mathbb{B}$ to one symbol. In this way, we reduce the r -th order RM code to an $(r-1)$ -th order RM code. For BSC, this mapping is simply the addition in $\mathbb{F}_2$. The sum $y_{/\mathbb{B}}(T)$ can be interpreted as an estimate of $c_{/\mathbb{B}}(T)$, where c is the transmitted (true) codeword. In other words,

$$\mathbb{P}(Y_{/\mathbb{B}}(T) = c_{/\mathbb{B}}(T)) > \mathbb{P}(Y_{/\mathbb{B}}(T) = c_{/\mathbb{B}}(T) \oplus 1),$$

where Y is the channel output random vector.

For general channels, we also want to estimate $c_{/\mathbb{B}}(T)$ based on the LLRs $(L(z), z \in T)$. More precisely, given $(y(z), z \in T)$, or equivalently given $(L(z), z \in T)$, we would like to calculate the following LLR:

$$L_{/\mathbb{B}}(T) := \ln \left(\frac{\mathbb{P}(Y(z) = y(z), z \in T | c_{/\mathbb{B}}(T) = 0)}{\mathbb{P}(Y(z) = y(z), z \in T | c_{/\mathbb{B}}(T) = 1)} \right).$$

We will make use of the following simple property of RM codes to calculate this LLR.

Lemma 2: Suppose that $r \geq 1$. Let C be a random codeword chosen uniformly from $\mathcal{RM}(m, r)$, and let $\mathbf{z}$ and $\mathbf{z}'$ be two distinct vectors in $\mathbb{E}$. Then the two coordinates $(C(\mathbf{z}), C(\mathbf{z}'))$ of the random codeword C have i.i.d. Bernoulli-1/2 distribution.

Proof: Define the following four sets

$$\begin{aligned}\mathcal{A}(0, 0) &:= \{c \in \mathcal{RM}(m, r) : c(\mathbf{z}) = c(\mathbf{z}') = 0\}, \\ \mathcal{A}(0, 1) &:= \{c \in \mathcal{RM}(m, r) : c(\mathbf{z}) = 0, c(\mathbf{z}') = 1\}, \\ \mathcal{A}(1, 0) &:= \{c \in \mathcal{RM}(m, r) : c(\mathbf{z}) = 1, c(\mathbf{z}') = 0\}, \\ \mathcal{A}(1, 1) &:= \{c \in \mathcal{RM}(m, r) : c(\mathbf{z}) = c(\mathbf{z}') = 1\}.\end{aligned}$$

To prove this lemma, we only need to show that $|\mathcal{A}(0, 0)| = |\mathcal{A}(0, 1)| = |\mathcal{A}(1, 0)| = |\mathcal{A}(1, 1)|$. Since RM code is linear and the all one vector is a codeword of RM codes, the marginal distribution of the coordinate $C(\mathbf{z})$ is Bernoulli-1/2 for every $\mathbf{z} \in \mathbb{E}$. Thus we have

$$\begin{aligned}|\mathcal{A}(0, 0)| + |\mathcal{A}(0, 1)| &= |\mathcal{A}(1, 0)| + |\mathcal{A}(1, 1)|, \\ |\mathcal{A}(0, 0)| + |\mathcal{A}(1, 0)| &= |\mathcal{A}(0, 1)| + |\mathcal{A}(1, 1)|.\end{aligned}\quad (11)$$

Now take $\mathbf{z} = (z_1, \dots, z_m)$ and $\mathbf{z}' = (z'_1, \dots, z'_m)$ such that $\mathbf{z} \neq \mathbf{z}'$. Then there exists $i \in [m]$ such that $z_i \neq z'_i$. Since we assume that $r \geq 1$, $\mathcal{RM}(m, r)$ contains the evaluation vector of the degree-1 monomial Z_i . We denote this evaluation vector as v , and we know that $v(\mathbf{z}) \neq v(\mathbf{z}')$. Without loss of generality, assume that $v(\mathbf{z}) = 0$ and $v(\mathbf{z}') = 1$. Then we have⁴ $\mathcal{A}(0, 0) + v \subseteq \mathcal{A}(0, 1)$, so $|\mathcal{A}(0, 0)| \leq |\mathcal{A}(0, 1)|$. Conversely, we also have $\mathcal{A}(0, 1) + v \subseteq \mathcal{A}(0, 0)$, so $|\mathcal{A}(0, 1)| \leq |\mathcal{A}(0, 0)|$. Therefore, $|\mathcal{A}(0, 1)| = |\mathcal{A}(0, 0)|$. Similarly, we can also show that $|\mathcal{A}(1, 1)| = |\mathcal{A}(1, 0)|$. Taking these into (11), we obtain that $|\mathcal{A}(0, 0)| = |\mathcal{A}(0, 1)| = |\mathcal{A}(1, 0)| = |\mathcal{A}(1, 1)|$, which completes the proof of the lemma. $\square$

Now we can calculate $L_{/\mathbb{B}}(T)$ using the following model: Suppose that S_1 and S_2 are i.i.d. Bernoulli-1/2 random variables, and we transmit them over two independent copies of the channel $W : \{0, 1\} \rightarrow \mathcal{W}$. The corresponding channel output random variables are denoted as X_1 and X_2 , respectively. Then for $x_1, x_2 \in \mathcal{W}$, we have (12), shown at the bottom of the page.

⁴For a set $\mathcal{A}$ and a vector v , we define the set $\mathcal{A} + v := \{a + v : a \in \mathcal{A}\}$.

Lemma 2 above allows us to replace x_1, x_2 with $(y(\mathbf{z}), \mathbf{z} \in T)$, and we obtain that

$$L_{/\mathbb{B}}(T) = \ln \left(\exp \left(\sum_{\mathbf{z} \in T} L(\mathbf{z}) \right) + 1 \right) - \ln \left(\sum_{\mathbf{z} \in T} \exp(L(\mathbf{z})) \right). \quad (13)$$

Now we are ready to present the decoding algorithm for general binary-input channels. In Algorithms 3–4 below, we still denote the decoding result of the $(r - 1)$ -th order RM code as $\hat{y}_{/\mathbb{B}}$ (see line 7 of Algorithm 3), where $\hat{y}_{/\mathbb{B}} = (\hat{y}_{/\mathbb{B}}(T), T \in \mathbb{E}/\mathbb{B})$ are indexed by the cosets $T \in \mathbb{E}/\mathbb{B}$, and we use $[\mathbf{z} + \mathbb{B}]$ to denote the coset containing $\mathbf{z}$ (see line 3 of Algorithm 4).

Algorithm 3 is very similar to Algorithm 1: From line 8 to line 10, we compare $\hat{L}(\mathbf{z})$ with the original $L(\mathbf{z})$. If the relative difference between these two is below the threshold θ for every $\mathbf{z} \in \mathbb{E}$, then the values of $L(\mathbf{z}), \mathbf{z} \in \mathbb{E}$ change very little in this iteration, and the algorithm reaches a “stable” state, so we can exit the for loop on line 2. In practice, we find that $\theta = 0.05$ works fairly well,⁵ and we still set the maximal number of iterations $N_{\max} = m/2$, which is the same as in Algorithm 1. On line 13, the algorithm simply produces the decoding result according to the LLR at each coordinate.

A few explanations of Algorithm 4: On line 3, we set $\text{cumuLLR}(\mathbf{z}) = \sum_{\mathbf{z}' \neq \mathbf{z}} \alpha(\mathbf{z}, \mathbf{z}') L(\mathbf{z}')$, where the coefficients $\alpha(\mathbf{z}, \mathbf{z}')$ can only be 1 or -1 . More precisely, $\alpha(\mathbf{z}, \mathbf{z}')$ is 1 if the decoding result of the corresponding $(r - 1)$ th order RM code at the coset $\{\mathbf{z}, \mathbf{z}'\}$ is 0, and $\alpha(\mathbf{z}, \mathbf{z}')$ is -1 if the decoding result at the coset $\{\mathbf{z}, \mathbf{z}'\}$ is 1. The reason behind this assignment is simple: The decoding result at the coset $\{\mathbf{z}, \mathbf{z}'\}$ is an estimate of $c(\mathbf{z}) \oplus c(\mathbf{z}')$. If $c(\mathbf{z}) \oplus c(\mathbf{z}')$ is more likely to be 0, then the sign of $L(\mathbf{z})$ and $L(\mathbf{z}')$ should be the same. Here $\text{cumuLLR}(\mathbf{z})$ serves as an estimate of $L(\mathbf{z})$ based on all the other $L(\mathbf{z}'), \mathbf{z}' \neq \mathbf{z}$, so we assign the coefficient $\alpha(\mathbf{z}, \mathbf{z}')$ to be 1. Otherwise, if $c(\mathbf{z}) \oplus c(\mathbf{z}')$ is more likely to be 1, then the sign of $L(\mathbf{z})$ and $L(\mathbf{z}')$ should be different, so we assign the coefficient $\alpha(\mathbf{z}, \mathbf{z}')$ to be -1 .

In Algorithms 3–4, we write the pseudo codes in a mathematical fashion for the ease of understanding. In Appendix C, we present another version of the RPA_RM function in a program language fashion; see Algorithm 9.

⁵The decoding error probability of this algorithm is non-increasing when we decrease the value of θ , and the running time of the algorithm increases when we decrease θ . Through simulations we find that the decoding error probability remains the same if we continue decreasing θ beyond 0.05. Therefore, $\theta = 0.05$ is a good choice in practice because smaller θ will only increase the running time and not decrease decoding error at all.

$$\begin{aligned}\ln \left(\frac{\mathbb{P}(X_1 = x_1, X_2 = x_2 | S_1 + S_2 = 0)}{\mathbb{P}(X_1 = x_1, X_2 = x_2 | S_1 + S_2 = 1)} \right) &= \ln \left(\frac{\mathbb{P}(X_1 = x_1, X_2 = x_2, S_1 + S_2 = 0)}{\mathbb{P}(X_1 = x_1, X_2 = x_2, S_1 + S_2 = 1)} \right) \\ &= \ln \left(\frac{\mathbb{P}(X_1 = x_1, X_2 = x_2, S_1 = 0, S_2 = 0) + \mathbb{P}(X_1 = x_1, X_2 = x_2, S_1 = 1, S_2 = 1)}{\mathbb{P}(X_1 = x_1, X_2 = x_2, S_1 = 0, S_2 = 1) + \mathbb{P}(X_1 = x_1, X_2 = x_2, S_1 = 1, S_2 = 0)} \right) \\ &= \ln \left(\frac{\frac{1}{4}W(x_1|0)W(x_2|0) + \frac{1}{4}W(x_1|1)W(x_2|1)}{\frac{1}{4}W(x_1|0)W(x_2|1) + \frac{1}{4}W(x_1|1)W(x_2|0)} \right) = \ln \left(\frac{\frac{W(x_1|0)W(x_2|0)}{W(x_1|1)W(x_2|1)} + 1}{\frac{W(x_1|0)}{W(x_1|1)} + \frac{W(x_2|0)}{W(x_2|1)}} \right) \\ &= \ln \left(\exp(\text{LLR}(x_1) + \text{LLR}(x_2)) + 1 \right) - \ln \left(\exp(\text{LLR}(x_1)) + \exp(\text{LLR}(x_2)) \right).\end{aligned}\quad (12)$$

Algorithm 3 The RPA_RM Decoding Function for General Binary-Input Memoryless Channels

Input: The LLR vector $(L(\mathbf{z}), \mathbf{z} \in \{0, 1\}^m)$; the parameters of the Reed-Muller code m and r ; the maximal number of iterations $N_{\max}$; the exiting threshold θ

Output: The decoded codeword $\hat{\mathbf{c}}$

```

1:  $\mathbb{E} := \{0, 1\}^m$ 
2: for  $j = 1, 2, \dots, N_{\max}$  do
3:    $L_{/\mathbb{B}_i} \leftarrow (L_{/\mathbb{B}_i}(T), T \in \mathbb{E}/\mathbb{B}_i)$  for  $i = 1, 2, \dots, 2^m - 1$ 
4:    $\triangleright L_{/\mathbb{B}_i}(T)$  is calculated from  $(L(\mathbf{z}), \mathbf{z} \in \mathbb{E})$  according
     to (13)
5:    $\hat{y}_{/\mathbb{B}_i} \leftarrow \text{RPA\_RM}(L_{/\mathbb{B}_i}, m - 1, r - 1, N_{\max}, \theta)$  for  $i =$ 
      $1, 2, \dots, 2^m - 1$ 
6:    $\triangleright$  If  $r = 2$ , then we use the Fast Hadamard Transform
     to decode the first-order RM code
7:    $\hat{L} \leftarrow \text{Aggregation}(L, \hat{y}_{/\mathbb{B}_1}, \hat{y}_{/\mathbb{B}_2}, \dots, \hat{y}_{/\mathbb{B}_{n-1}})$ 
8:   if  $|\hat{L}(\mathbf{z}) - L(\mathbf{z})| \leq \theta |L(\mathbf{z})|$  for all  $\mathbf{z} \in \mathbb{E}$  then  $\triangleright$  The
     algorithm reaches a stable point
9:     break
10:  end if
11:   $L \leftarrow \hat{L}$ 
12: end for
13:  $\hat{\mathbf{c}}(\mathbf{z}) \leftarrow \mathbb{1}[L(\mathbf{z}) < 0]$  for each  $\mathbf{z} \in \mathbb{E}$ 
14: return  $\hat{\mathbf{c}}$ 

```

Algorithm 4 The Aggregation Function for General Binary-Input Memoryless Channels

Input: $L, \hat{y}_{/\mathbb{B}_1}, \hat{y}_{/\mathbb{B}_2}, \dots, \hat{y}_{/\mathbb{B}_{n-1}}$

Output: $\hat{L}$

```

1: Initialize  $(\text{cumuLLR}(\mathbf{z}), \mathbf{z} \in \{0, 1\}^m)$  as an all-zero vector
   indexed by  $\mathbf{z} \in \{0, 1\}^m$ 
2:  $n \leftarrow 2^m$ 
3:  $\text{cumuLLR}(\mathbf{z}) \leftarrow \sum_{i=1}^{n-1} ((1 - 2\hat{y}_{/\mathbb{B}_i}([\mathbf{z} + \mathbb{B}_i]))L(\mathbf{z} \oplus \mathbf{z}_i))$  for
   each  $\mathbf{z} \in \{0, 1\}^m$ 
4:    $\triangleright \mathbf{z}_i$  is the nonzero element in  $\mathbb{B}_i$ 
5:    $\triangleright \hat{y}_{/\mathbb{B}_i}$  is the decoded codeword, so  $\hat{y}_{/\mathbb{B}_i}([\mathbf{z} + \mathbb{B}_i])$  is
     either 0 or 1
6:  $\hat{L}(\mathbf{z}) \leftarrow \frac{\text{cumuLLR}(\mathbf{z})}{n-1}$  for each  $\mathbf{z} \in \{0, 1\}^m$ 
7: return  $\hat{L}$ 

```

Following the same proof of Proposition 1, we have the following result:

Proposition 3: The complexity of Algorithm 3 is $O(n^r \log n)$ in sequential implementation and $O(n^2)$ in parallel implementation with $O(n^r)$ processors.

In Section V, we present an accelerated version of the RPA algorithm for high-rate RM codes, and in Section VI-C, we further discuss other possible options to reduce the computation time by using fewer subspaces in the projection step.

Similarly to Proposition 2, we can also show that the decoding error probability of Algorithm 3 is independent of the transmitted codeword for binary-input memoryless symmetric (BMS) channels.

Definition 3 (BMS channel): We say that a memoryless channel $W : \{0, 1\} \rightarrow \mathcal{W}$ is a BMS channel if there is a

permutation π of the output alphabet $\mathcal{W}$ such that $\pi^{-1} = \pi$ and $W(x|1) = W(\pi(x)|0)$ for all $x \in \mathcal{W}$.

Proposition 4: Let $W : \{0, 1\} \rightarrow \mathcal{W}$ be a BMS channel. Let c_1 and c_2 be two codewords of $\mathcal{RM}(m, r)$. Let Y_1 and Y_2 be the (random) channel outputs of transmitting c_1 and c_2 over $n = 2^m$ independent copies of W , respectively. Let $L^{(1)}$ and $L^{(2)}$ be the LLR vectors corresponding to Y_1 and Y_2 , respectively.⁶ Then for any $c_1, c_2 \in \mathcal{RM}(m, r)$, we have

$$\begin{aligned} & \mathbb{P}(\text{RPA_RM}(L^{(1)}, m, r, N_{\max}, \theta) \neq c_1) \\ &= \mathbb{P}(\text{RPA_RM}(L^{(2)}, m, r, N_{\max}, \theta) \neq c_2). \end{aligned}$$

The proof is given in Appendix B. Similarly to Proposition 2, this proposition is also very useful for simulations because we can simply transmit the all-zero codeword over the BMS channel W to measure the decoding error probability.

In the last part of this section, we present the list decoding version of the RPA_RM function. The main idea is already explained in Section II-A. Here we only write down the pseudo code of the list decoding version. Note that the purpose of line 8 is to make sure that $\hat{\mathbf{c}}^{(u)}$ is a codeword of RM code, which is not always true for the decoding result of the RPA_RM function.

Finally, we present the following proposition on the memory requirement for sequential implementation of RPA decoder. A remarkable thing here is that the memory requirement for the list decoding version of RPA algorithm is $5n$, which is independent of the list size, in contrast to SCL decoder of polar codes.

Proposition 5: The memory needed for sequential implementation of the RPA decoder without list decoding is no more than $4n$, and the memory needed for sequential implementation of the RPA decoder with list decoding is no more than $5n$, where n is the code length. Note that the memory requirement for list decoding version does not depend on the list size.

Proof: As we mentioned above, Algorithm 3 is written in compact fashion for the ease of understanding, but it is not space-efficient in practical implementation. The version that we really implemented in practice and used for simulations is Algorithm 9 in Appendix C, and our analysis of space complexity is based on Algorithm 9.

The most important difference between Algorithm 9 and Algorithm 3 is that in Algorithm 3 we first finish all the recursive decoding and then perform the aggregation step; while in Algorithm 9 the recursive decoding step and the aggregation step are interleaved together, and in this way we can save huge amount of memory compared to Algorithm 3.

We start with RPA decoder without list decoding, and we prove by induction on r , the order of the RM code. For the base case of $r = 1$, the claim clearly holds. Now assume that the claim holds for all RM codes with order $< r$ and we prove it for order r . In Algorithm 9, we need n floating number positions to store the LLR vector and another n floating number positions to store the cumuLLR vector. Then we project onto the cosets of each one-dimensional subspace sequentially. For each projected codeword, we need to decode

⁶ Y_1 and Y_2 are random vectors, and the randomness comes from the channel noise. As a result, $L^{(1)}$ and $L^{(2)}$ are also random vectors.

Algorithm 5 The RPA_LIST Decoding Function for General Binary-Input Memoryless Channels

Input: The LLR vector $(L(\mathbf{z}), \mathbf{z} \in \{0, 1\}^m)$; the parameters of the Reed-Muller code m and r ; the maximal number of iterations $N_{\max}$; the exiting threshold θ ; the list size 2^t

Output: The decoded codeword $\hat{c}$

```

1:  $\tilde{L} \leftarrow L$ 
2:  $(\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_t) \leftarrow$  indices of the  $t$  smallest entries in  $(|L(\mathbf{z})|, \mathbf{z} \in \{0, 1\}^m)$ 
3:  $\triangleright \mathbf{z}_i \in \{0, 1\}^m$  for all  $i = 1, 2, \dots, t$ 
4:  $L_{\max} \leftarrow 2 \max(|L(\mathbf{z})|, \mathbf{z} \in \{0, 1\}^m)$ 
5: for each  $\mathbf{u} \in \{L_{\max}, -L_{\max}\}^t$  do
6:    $(L(\mathbf{z}_1), L(\mathbf{z}_2), \dots, L(\mathbf{z}_t)) \leftarrow \mathbf{u}$ 
7:    $\hat{c}^{(\mathbf{u})} \leftarrow \text{RPA\_RM}(L, m, r, N_{\max}, \theta)$ 
8:    $\hat{c}^{(\mathbf{u})} \leftarrow \text{Reedsdecoder}(\hat{c}^{(\mathbf{u})})$   $\triangleright$  Reedsdecoder is the classical decoding algorithm in [2]
9: end for
10:  $\mathbf{u}^* \leftarrow \arg\max_{\mathbf{u}} \sum_{\mathbf{z} \in \{0, 1\}^m} ((-1)^{\hat{c}^{(\mathbf{u})}(\mathbf{z})} \tilde{L}(\mathbf{z}))$ 
11:  $\triangleright$  This follows from (9). Maximization is over  $\mathbf{u} \in \{L_{\max}, -L_{\max}\}^t$ 
12:  $\hat{c} \leftarrow \hat{c}^{(\mathbf{u}^*)}$ 
13: return  $\hat{c}$ 

```

a RM code with length $n/2$ and order $r - 1$. By induction hypothesis, this take $4 * n/2 = 2n$ floating number positions. Therefore in total we need $n + n + 2n = 4n$ floating number positions. This establishes the inductive step and completes the proof for the non-list-decoding version.

The memory requirement for list decoding version follows directly from that of the vanilla version: Since we perform list decoding sequentially, i.e., we only decode one list at a time, the only extra memory we need in the list decoding version is the n floating number positions that is used to store currently best known decoding result. Therefore, the space complexity for the list decoding version is $5n$. $\square$

V. SIMPLIFIED RPA ALGORITHM FOR HIGH RATE RM CODES

In this section, we provide some simplified versions of the RPA decoder, which significantly accelerate the decoding process while maintaining the same (nearly optimal) decoding error probability for certain RM codes with rate > 0.5 .

As mentioned in the previous section, we can accelerate the decoding algorithm by using fewer subspaces in the projection step. Moreover, instead of using one-dimensional subspaces, in this section we propose to use a selected subsets of two-dimensional subspaces in the projection step. In particular, we only project onto the $\binom{m}{2}$ two-dimensional subspaces spanned by two standard basis vectors of $\mathbb{E}$. The standard basis vector of $\mathbb{E}$ are $\mathbf{e}^{(1)}, \dots, \mathbf{e}^{(m)}$, where $\mathbf{e}^{(i)}$ is defined as the vector with 1 in the i th position and 0 everywhere else. Then we write the $\binom{m}{2}$ two-dimensional subspaces as $\{\mathbb{B}_{i,j} : 1 \leq i < j \leq m\}$, where

$$\mathbb{B}_{i,j} := \text{span}(\mathbf{e}^{(i)}, \mathbf{e}^{(j)}).$$

Note that projection onto cosets of two-dimensional subspaces is different from onto that of one-dimensional

subspaces: In the one-dimensional case, each coset only contains two coordinates, and we only need to combine the LLR of two coordinates to obtain the LLR of the coset, as we did in (13). In the two-dimensional case, each coset contains four coordinates, and we need to combine the LLR of four coordinates to obtain the LLR of the coset. Fortunately, for any RM code with order $r \geq 2$, we can use exactly the same idea in the proof of Lemma 2 to show that any four coordinates in a coset of a two-dimensional subspace are also independent; see the explanation in Remark 1 below. Therefore, we obtain the following counterpart of (13) for a coset T of two-dimensional subspace assuming that $T = \{\mathbf{z}^{(1)}, \mathbf{z}^{(2)}, \mathbf{z}^{(3)}, \mathbf{z}^{(4)}\}$:

$$\begin{aligned}
 L_{/\mathbb{B}}(T) = & \ln \left(\exp \left(\sum_{i=1}^4 L(\mathbf{z}^{(i)}) \right) \right. \\
 & \left. + \sum_{1 \leq i < j \leq 4} \exp(L(\mathbf{z}^{(i)}) + L(\mathbf{z}^{(j)})) + 1 \right) \\
 & - \ln \left(\sum_{i=1}^4 \exp(L(\mathbf{z}^{(i)})) + \sum_{i=1}^4 \exp \left(\sum_{j \in [4] \setminus \{i\}} L(\mathbf{z}^{(j)}) \right) \right).
 \end{aligned} \tag{14}$$

Remark 1: It is well known that for a linear code, if there is a codeword taking value 1 at a certain coordinate, then the number of codewords taking value 1 at this coordinate is the same as the number of codewords taking value 0 at this coordinate. This follows directly from the linearity of the code. The proof of Lemma 2 follows from the same idea: By the linearity of code, we only need to show that for two distinct coordinates, there are different codewords in RM codes that take all four possible values $(0, 0), (0, 1), (1, 0), (1, 1)$ at these two coordinates, and this follows by noting that (i) any two distinct coordinates form a coset of a one-dimensional subspace; (ii) by definition of RM codes, restricting RM codes with order $r \geq 1$ on such cosets gives us $\mathcal{RM}(1, 1)$, which contains all 4 binary vectors of length 2. Now in the case of two-dimensional subspace, we still use the same reasoning: By linearity of the code, we only need to show that for any 4 coordinates that form a coset of a 2-dimensional subspace, there are different codewords in RM codes with order $r \geq 2$ that take all 2^4 possible values $\{0, 1\}^4$ at these four coordinates. This again follows by noting that restricting RM codes with order $r \geq 2$ on such cosets gives us $\mathcal{RM}(2, 2)$, which contains all 16 binary vectors of length 4.

After projecting $\mathcal{RM}(m, r)$ onto the cosets of these two-dimensional subspaces, we will obtain RM codes with parameters $m - 2$ and $r - 2$, as proved in Lemma 1. After decoding these $\binom{m}{2}$ projected codes $\mathcal{RM}(m - 2, r - 2)$, we obtain $\{\hat{y}_{/\mathbb{B}_{i,j}} : 1 \leq i < j \leq m\}$, where $\hat{y}_{/\mathbb{B}_{i,j}} = (\hat{y}_{/\mathbb{B}_{i,j}}(T), T \in \mathbb{E}/\mathbb{B}_{i,j})$. Now we are ready to go to the aggregation step using both the recursive decoding result $\{\hat{y}_{/\mathbb{B}_{i,j}} : 1 \leq i < j \leq m\}$ and the original LLR vector L . In particular, when decoding $c(\mathbf{z})$, the relevant coordinate in $\hat{y}_{/\mathbb{B}_{i,j}}$ is $\hat{y}_{/\mathbb{B}_{i,j}}([z + \mathbb{B}_{i,j}])$, where $[z + \mathbb{B}_{i,j}]$ is the coset of $\mathbb{B}_{i,j}$ that contains $\mathbf{z}$. Now suppose that the other three vectors in $[z + \mathbb{B}_{i,j}]$ apart from $\mathbf{z}$ itself are $\mathbf{z}^{(1)}, \mathbf{z}^{(2)}, \mathbf{z}^{(3)}$. Then from $\hat{y}_{/\mathbb{B}_{i,j}}([z + \mathbb{B}_{i,j}])$ and $L(\mathbf{z}^{(1)}), L(\mathbf{z}^{(2)}), L(\mathbf{z}^{(3)})$, we obtain the

following estimate of the LLR of $c(\mathbf{z})$:

$$\begin{aligned} \text{Est}_{i,j}(\mathbf{z}) &= \ln \left(\exp \left(\sum_{i=1}^3 L(\mathbf{z}^{(i)}) \right) + \sum_{i=1}^3 \exp(L(\mathbf{z}^{(i)})) \right) \\ &\quad - \ln \left(\sum_{i=1}^3 \exp \left(\sum_{j \in [3] \setminus \{i\}} L(\mathbf{z}^{(j)}) \right) + 1 \right) \\ &\quad \text{if } \hat{y}_{/\mathbb{B}_{i,j}}([\mathbf{z} + \mathbb{B}_{i,j}]) = 0, \\ \text{Est}_{i,j}(\mathbf{z}) &= -\ln \left(\exp \left(\sum_{i=1}^3 L(\mathbf{z}^{(i)}) \right) + \sum_{i=1}^3 \exp(L(\mathbf{z}^{(i)})) \right) \\ &\quad + \ln \left(\sum_{i=1}^3 \exp \left(\sum_{j \in [3] \setminus \{i\}} L(\mathbf{z}^{(j)}) \right) + 1 \right) \\ &\quad \text{if } \hat{y}_{/\mathbb{B}_{i,j}}([\mathbf{z} + \mathbb{B}_{i,j}]) = 1. \end{aligned} \quad (15)$$

We calculate such an estimate for all pairs of (i, j) such that $1 \leq i < j \leq m$. Then finally we update the LLR of $c(\mathbf{z})$ as the average of these $\binom{m}{2}$ estimates, as follows:

$$\hat{L}(\mathbf{z}) = \frac{1}{\binom{m}{2}} \sum_{1 \leq i < j \leq m} \text{Est}_{i,j}(\mathbf{z}).$$

Finally, as in all the previous sections, we iterate this decoding procedure a few times for the LLR vector to converge to a stable value.

We call the decoding algorithm proposed in this section the *Simplified_RPA* algorithm, as opposed to the normal RPA algorithm proposed in the previous section. Note here that in the recursive decoding procedure, i.e., when we decode $\mathcal{RM}(m-2, r-2)$, we still use this simplified version of RPA algorithm instead of doing full projection step. Since each time we reduce r by 2, if the original r is even then we will not reach the first-order RM codes. In this case, we use the normal RPA decoder when we reach the second-order RM codes. In Algorithm 6 and Algorithm 7 we provide pseudo-codes for the *Simplified_RPA* algorithm. Note that in line 7–8 of Algorithm 6, we distinguish between the cases of r being even and r being odd: For even r , eventually we will need to decode a second-order RM code using the normal RPA decoder while for odd r , we only need to decode first-order RM code in the final recursive step. As we will show in Section VI (see Fig. 2), by applying the list decoding version of the *Simplified_RPA* algorithm, we can decode $\mathcal{RM}(7, 4)$ and $\mathcal{RM}(8, 5)$ with list size no larger than 8 such that the decoding error probability is the same as that of ML decoder. Moreover, it runs even faster than decoding lower rate codes such as $\mathcal{RM}(8, 3)$; see Table I.

VI. SIMULATION RESULTS

A. Comparison With Polar Codes

We run our decoding algorithm for second and third order Reed-Muller codes with code length 256, 512 and 1024 over AWGN channels and BSCs, and we compare its performance with the recent algorithms for polar codes with the same length and dimension. We compare to two versions of polar codes: Polar codes with optimal CRC size and polar codes without

Algorithm 6 The Simplified_RPA Decoding Function

Input: The LLR vector $(L(\mathbf{z}), \mathbf{z} \in \{0, 1\}^m)$; the parameters of the Reed-Muller code m and r ; the maximal number of iterations $N_{\max}$; the exiting threshold θ

Output: The decoded codeword $\hat{c}$

```

1:  $\mathbb{E} := \{0, 1\}^m$ 
2: for  $j = 1, 2, \dots, N_{\max}$  do
3:    $L_{/\mathbb{B}_{i,j}} \leftarrow (L_{/\mathbb{B}_{i,j}}(T), T \in \mathbb{E}_{/\mathbb{B}_{i,j}})$  for  $1 \leq i < j \leq m$ 
4:    $\triangleright L_{/\mathbb{B}_{i,j}}(T)$  is calculated according to (14)
5:    $\hat{y}_{/\mathbb{B}_{i,j}} \leftarrow \text{Simplified\_RPA}(L_{/\mathbb{B}_{i,j}}, m-2, r-2, N_{\max}, \theta)$  for  $1 \leq i < j \leq m$ 
6:    $\triangleright$  If  $r = 3$ , then we use the Fast Hadamard Transform to decode the first-order RM code
7:    $\triangleright$  If  $r = 4$ , then we use the normal RPA algorithm to decode the second-order RM code
8:    $\hat{L} \leftarrow \text{Simp\_Aggregation}(L, \{\hat{y}_{/\mathbb{B}_{i,j}} : 1 \leq i < j \leq m\})$ 
9:   if  $|\hat{L}(\mathbf{z}) - L(\mathbf{z})| \leq \theta |L(\mathbf{z})|$  for all  $\mathbf{z} \in \mathbb{E}$  then  $\triangleright$  The algorithm reaches a stable point
10:    break
11:  end if
12:   $L \leftarrow \hat{L}$ 
13: end for
14:  $\hat{c}(\mathbf{z}) \leftarrow \mathbb{1}[L(\mathbf{z}) < 0]$  for each  $\mathbf{z} \in \mathbb{E}$ 
15: return  $\hat{c}$ 
```

Algorithm 7 The Simp_Aggregation Function in the Simplified_RPA Algorithm

Input: $L, \{\hat{y}_{/\mathbb{B}_{i,j}} : 1 \leq i < j \leq m\}$

Output: $\hat{L}$

```

1: Calculate  $\text{Est}_{i,j}(\mathbf{z})$  from  $L$  and  $\{\hat{y}_{/\mathbb{B}_{i,j}} : 1 \leq i < j \leq m\}$  according to (15)
2:  $\hat{L}(\mathbf{z}) \leftarrow \frac{1}{\binom{m}{2}} \sum_{1 \leq i < j \leq m} \text{Est}_{i,j}(\mathbf{z})$  for each  $\mathbf{z} \in \{0, 1\}^m$ 
3: return  $\hat{L}$ 
```

TABLE I

COMPARISON OF DECODING TIME BETWEEN RM CODES AND POLAR CODES. $P(m, r)$ DENOTES POLAR CODES WITH THE SAME LENGTH AND DIMENSION AS $\mathcal{RM}(m, r)$

$\mathcal{RM}(7, 2)$ 1ms	$P(7, 2)$ 7ms	$\mathcal{RM}(7, 3)$ 26ms	$P(7, 3)$ 15ms	$\mathcal{RM}(7, 4)$ 6ms	$P(7, 4)$ 23ms
$\mathcal{RM}(8, 2)$ 4.3ms	$P(8, 2)$ 17ms	$\mathcal{RM}(8, 3)$ 236ms	$P(8, 3)$ 40ms	$\mathcal{RM}(8, 4)$ 5.9s	$P(8, 4)$ 64ms
$\mathcal{RM}(8, 5)$ 14ms	$P(8, 5)$ 82ms	$\mathcal{RM}(9, 2)$ 18.2ms	$P(9, 2)$ 41ms	$\mathcal{RM}(10, 2)$ 76.7ms	$P(10, 2)$ 95ms

CRC, and we use the Successive Cancellation List (SCL) decoder introduced by Tal and Vardy [20] as the decoder, where we set list size to be 32. Note that SCL decoder with list size 32 is one of the most widely used decoders for polar codes.

The simulation results for AWGN channels are plotted in Figure 2, where the number of Monte Carlo

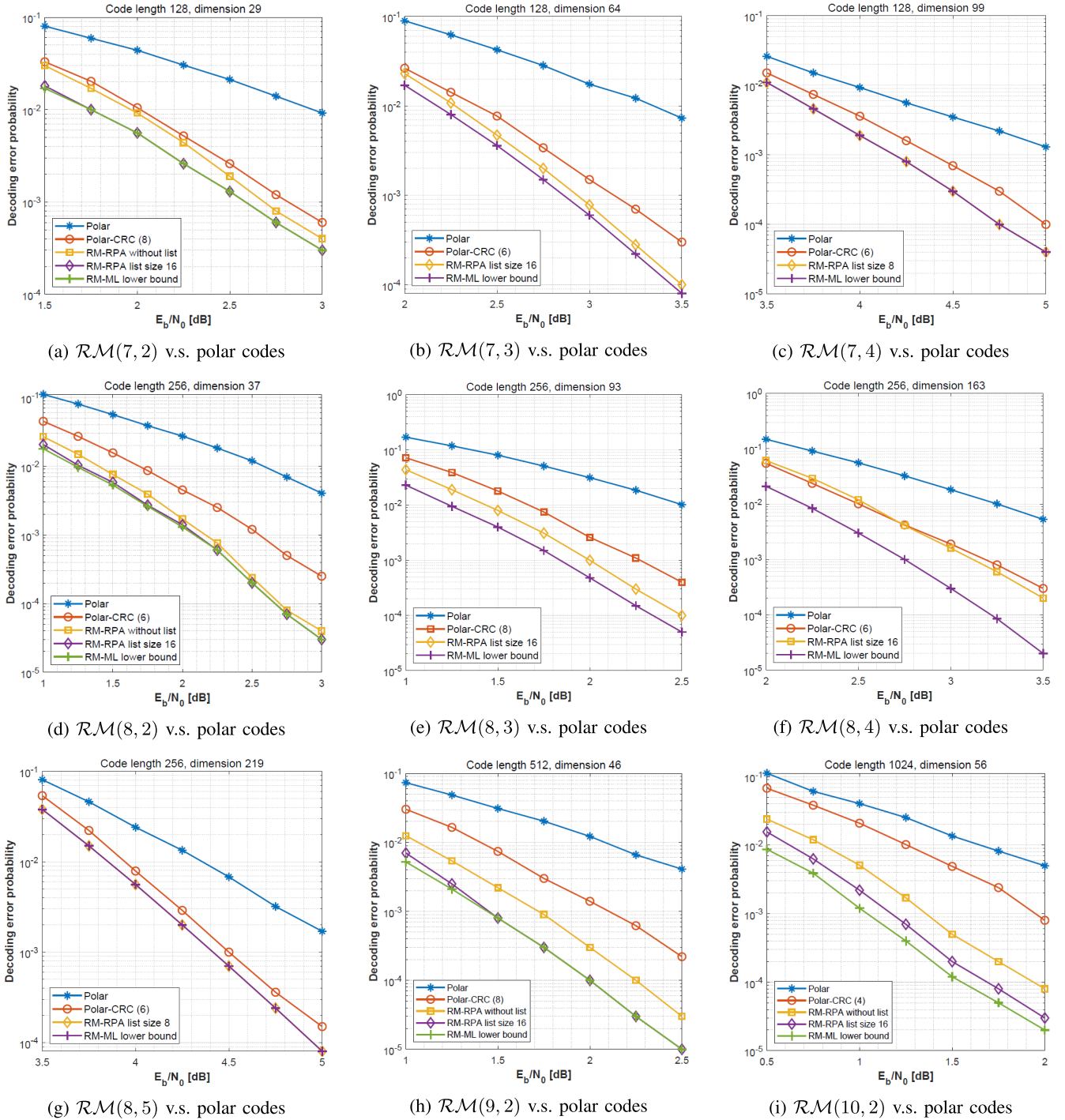


Fig. 2. Comparison between Reed-Muller codes and polar codes over AWGN channels. For $\mathcal{RM}(7,4)$ and $\mathcal{RM}(8,5)$, we use the Simplified_RPA algorithm proposed in Section V, and for all the other RM codes, we use the normal RPA algorithm proposed in Section IV. For polar codes with or without CRC, we always use SCL decoder with list size 32. For polar codes with CRC, we test various choices of CRC length and choose the optimal one that gives the best performance. The number in the bracket after “Polar-CRC” is the optimal CRC length that we use.

trials is 10^5 . We provide the simulation results for all RM codes with length 128 and 256, including $\mathcal{RM}(7,2)$, $\mathcal{RM}(7,3)$, $\mathcal{RM}(7,4)$, $\mathcal{RM}(8,2)$, $\mathcal{RM}(8,3)$, $\mathcal{RM}(8,4)$, $\mathcal{RM}(8,5)$. This should give a complete picture of the performance of our decoder for all code rates. Note that we skipped $\mathcal{RM}(7,5)$ and $\mathcal{RM}(8,6)$ because they are extended Hamming codes, and optimal decoders are well known for these two codes. Moreover, for certain cases the list decoding version of RPA decoding algorithm has almost the

same performance as the Maximal Likelihood (ML) decoder for RM codes.⁷ The performance improvement is thus in agreement with the advantages of RM codes over polar codes

⁷We use the method in [14], [16] to measure the ML lower bound: Whenever our decoder outputs a wrong codeword, we compare the posterior probability of the decoded word and that of the correct codeword. Most of the time the posterior probability of the decoded word is larger, which means that even an ML decoder will make a mistake in this case. Note that this method was also used in [20].

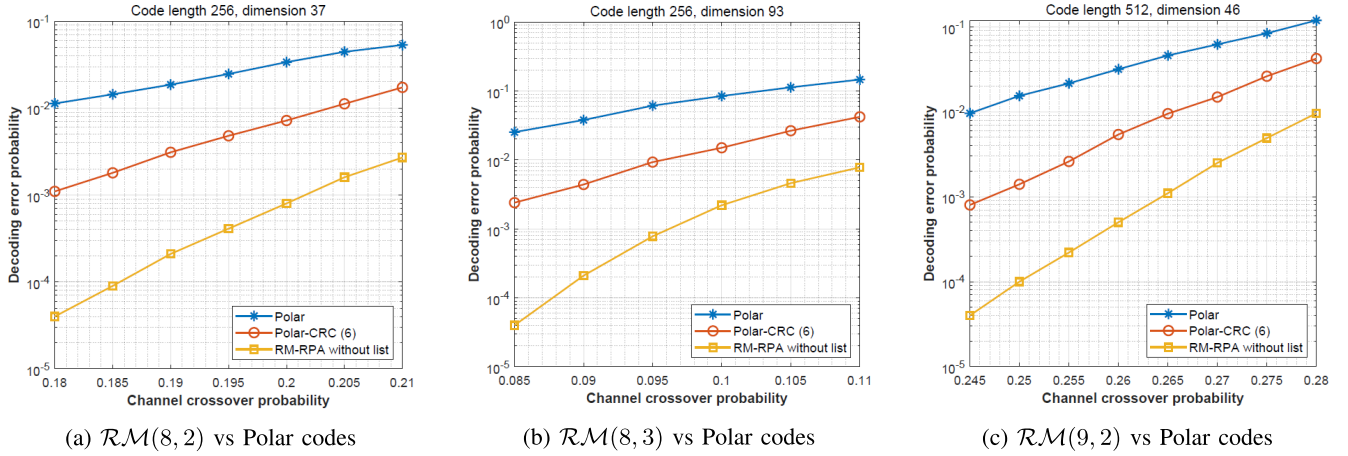


Fig. 3. Comparison between Reed-Muller codes and polar codes over BSC channels. For RM codes we use the RPA decoder in Algorithm 1 **without list decoding**. For polar codes, no matter with or without CRC, we always use SCL decoder with list size 32.

under ML decoding [5]. See Section VI-B for comparisons with Dumer's recursive decoding algorithm [14]–[16], which is the best known decoder in the literature for RM codes over AWGN channels. Note also that the algorithm in [19] only applies to codes with very short code length (no larger than 128) due to complexity constraints.

For the BSC channel, the simulation results are plotted in Figure 3. The number of Monte Carlo trials is 10^5 . We also tested in this case all the previous decoding algorithms known for RM codes, including Reed's algorithm [2] and the algorithm from Saptharishi-Shpilka-Volk [17]. For these two algorithms, the decoding error probability exceeds 0.1 for the tested parameters, so we did not include them in Figure 3 as they would not fit. See Section VI-B for comparisons with the Sidel'nikov-Pershakov algorithm [11] and its variations [12], [13]. From Figure 3, we can clearly see that the new decoding algorithm for RM codes significantly outperforms the SCL decoder for CRC-aided polar codes.

We also compare the running time of our decoder and the SCL decoder for polar codes. For polar codes, we use techniques from two accelerated version [26], [27] of the SCL decoder (in particular the "min-sum approximation" in [26]) so that we can achieve a much smaller running time than the original version of SCL decoder while maintaining almost the same decoding error probability. The results are listed in Table I. We can see that for second order RM codes as well as the high-rate RM codes where we use the Simplified_RPA algorithm to decode, our decoder is always faster than the SCL decoder for polar codes with the same parameters. However, for third order RM codes, our decoder is slower than the SCL decoder; see Fig 2 for decoding error probability and Table I for running time.

B. Comparison With Previous Decoding Algorithms of RM Codes

We first compare with the decoding algorithm proposed by Sidel'nikov and Pershakov [11], which was later improved/modified in [12], [13]. When decoding the second-order RM codes, the RPA decoding algorithm has

some high-level similarity with the decoding algorithms in [11]–[13] in the sense that the first step in all these algorithms is to project the received word y onto the cosets of all the $n-1$ one-dimensional subspaces and decode the projected first-order RM codewords to obtain $\hat{y}_{/\mathbb{B}_1}, \hat{y}_{/\mathbb{B}_2}, \dots, \hat{y}_{/\mathbb{B}_{n-1}}$. However, the next steps in [11]–[13] are quite different from the RPA decoding algorithm and result in a worse performance than the RPA algorithm. More precisely, the main differences are:

- The decoding algorithms in [12], [13] only work for the second order RM codes. For higher-order RM codes, the decoding algorithm proposed in [11] is completely different from the RPA algorithm, and their performance is much worse than the RPA algorithm; see Fig. 4(c).
- For second order RM codes, after the projection step, the RPA algorithm make use of both the decoding results of the projected codewords $\hat{y}_{/\mathbb{B}_1}, \hat{y}_{/\mathbb{B}_2}, \dots, \hat{y}_{/\mathbb{B}_{n-1}}$ and the original received word y to obtain the final decoding results while the algorithms in [11]–[13] only make use of $\hat{y}_{/\mathbb{B}_1}, \hat{y}_{/\mathbb{B}_2}, \dots, \hat{y}_{/\mathbb{B}_{n-1}}$ to obtain the coefficients of all the degree-2 monomials⁸ in the final decoding results. As discussed above, the projected codewords are more noisy than the original received words y . As a consequence, the performance of the algorithms in [11]–[13] is worse than that of the RPA algorithm; see Fig. 4(a),(b).
- The RPA algorithm uses $\hat{y}_{/\mathbb{B}_1}, \hat{y}_{/\mathbb{B}_2}, \dots, \hat{y}_{/\mathbb{B}_{n-1}}$ together with the original received word y to correct errors bitwise in the original received word y while the algorithms in [11]–[13] use $\hat{y}_{/\mathbb{B}_1}, \hat{y}_{/\mathbb{B}_2}, \dots, \hat{y}_{/\mathbb{B}_{n-1}}$ to correct errors wordwise among themselves.

In Fig. 4, we compare the RPA algorithm with the algorithms in [11]–[13] for decoding Reed-Muller codes over AWGN and BSC channels. Note that there are two parameters s and h in the Sidel'nikov-Pershakov algorithm, where s is the list size of decoding each projected codeword, and h is the number of iterations when decoding the projected codewords. In our simulations, we set $s = 4$ and $h = 3$ since larger values of s and h will not further improve the performance.

⁸Recall Definition 1 and the discussion following it.

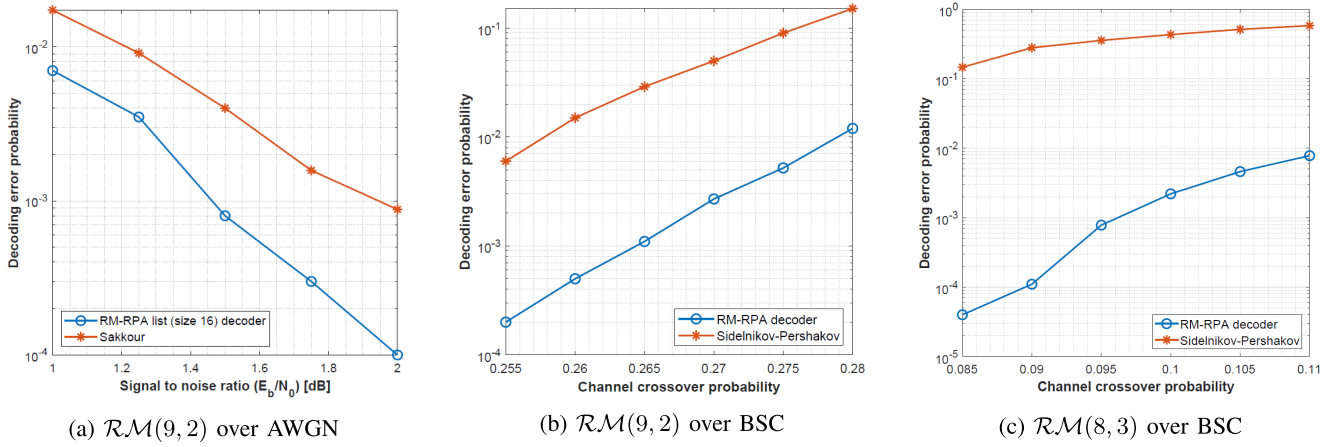


Fig. 4. Comparison between the RPA algorithm and the algorithms in [11]–[13] for decoding Reed-Muller codes over AWGN and BSC channels. The curve with legend “Sakkour” is the performance of the algorithm in [12], [13], and the curves with legend “Sidelnikov-Pershakov” represent the performance of the algorithms in [11].

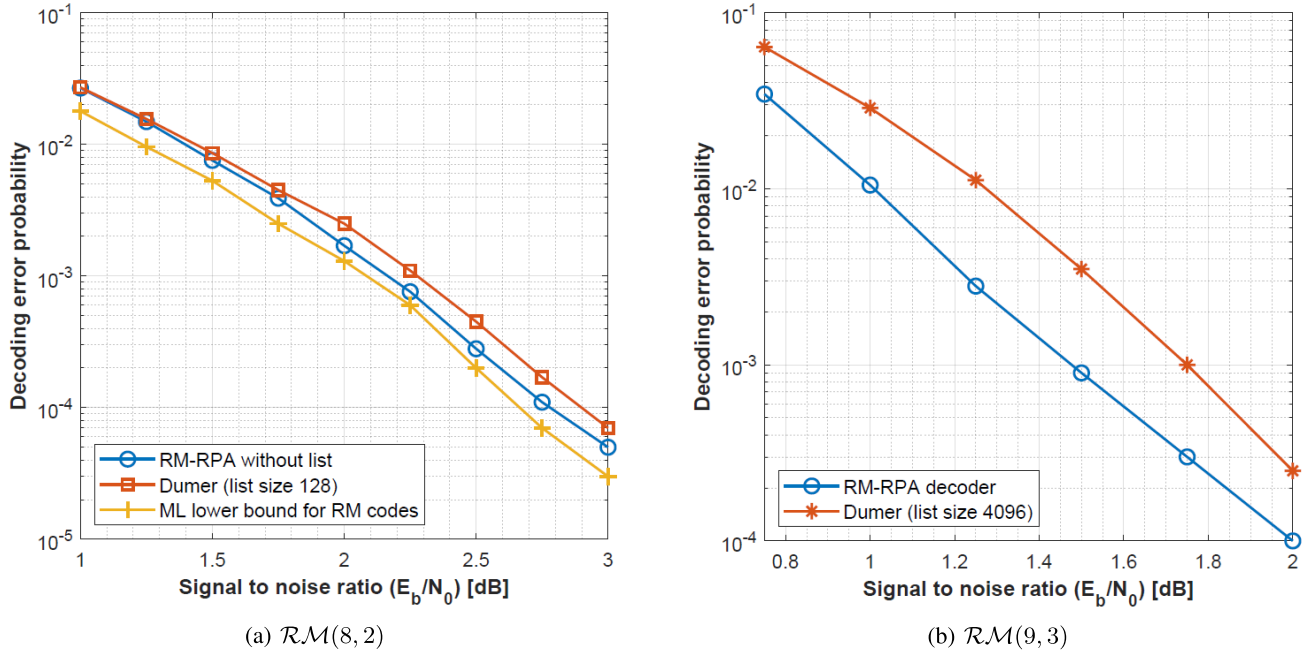


Fig. 5. Comparison between the RPA decoding algorithm **without list** and Dumer's recursive list decoding algorithm (the algorithm described in Section III of [16]) for decoding Reed-Muller codes over AWGN channels.

Next we compare the RPA algorithm with Dumer's recursive list decoding algorithm [14]–[16]. Dumer's list decoding algorithm provides a tradeoff between the decoding error probability and the decoding time. More precisely, if we set the list size to be large enough (e.g., exponential in n), then we can achieve the same performance as the maximal likelihood decoder, but we will also need exponential running time. If we choose small list size, then the algorithm runs fast but the decoding error will deteriorate.

In our simulations, we use the RPA algorithm and Dumer's algorithm to decode RM codes over AWGN channels, and we find that the decoding error probability of RPA is slightly better (smaller) than Dumer's algorithm, but the running time of RPA is typically larger. We have tested two cases $\mathcal{RM}(8, 2)$

and $\mathcal{RM}(9, 3)$, and the performance is given in Fig. 5. For $\mathcal{RM}(8, 2)$, the running time of our algorithm is 4.3ms, and the running time of Dumer's algorithm is 0.85ms. For $\mathcal{RM}(9, 3)$, the running time of our algorithm is 3s, and the running time of Dumer's algorithm is 0.14s.

In [19], simulation results are presented for $\mathcal{RM}(7, 3)$. Their results are based on applying belief propagation to all minimum weight parity checks. This does seem indirectly related to using all first-order RM subcodes to decode. For $\mathcal{RM}(7, 3)$, the decoding complexities of these two approaches are also similar. For RPA, each of 127×63 projections takes roughly 32×5 operations to decode, giving 1.2M operations per iteration. For the algorithm in [19], there are 94448 minimum weight parity checks of weight 16 giving roughly 1.5M

operations per iteration. It turns out that for $\mathcal{RM}(7,3)$, both the performance and the running time of RPA decoder are similar to the algorithm in [19].

We also note that in [28], an algorithm with near-ML performance was also provided for $\mathcal{RM}(7,3)$.

C. Parallelization and Acceleration

Another important advantage of the new decoding algorithm for RM codes over the SCL decoder for polar codes is that our algorithm naturally allows parallel implementation while the SCL decoder is not parallelizable. The key step in our algorithm for decoding a codeword of $\mathcal{RM}(r, m)$ is to decode the quotient space codes which are in $\mathcal{RM}(r-1, m-1)$ codes, and each of these can be decoded in parallel. Such a parallel structure is crucial to achieving high throughput and low latency.

Another way to accelerate the algorithm is to use only certain “voting sets”: In the projection step, we can take a subset of one-dimensional subspaces instead of all the one-dimensional subspaces. Then we still use recursive decoding followed by the aggregation step. In this way, we decode fewer $\mathcal{RM}(r-1, m-1)$ codes, and if the voting sets were chosen properly, we would obtain a similar decoding error probability with shorter running time. Note that in Section V we already gave a concrete choice of voting set in Algorithm 6, which indeed accelerates the decoding of high-rate RM codes with nearly-ML decoding error probability. At the same time, there might be other good voting sets to explore.

D. Comparison With the Meta Converse Bound for Optimal Codes [29], [30]

We compared with upper bound from Corollary 39 and lower bound from Theorem 40 in [30]. More precisely, we provide the target error probability, the noise parameter of the channel, and the code dimension, then Corollary 39 and Theorem 40 in [30] give us upper and lower bound on the (optimal) code length. We found that $\mathcal{RM}(8,2)$ is nearly optimal in terms of code length in the sense that the lower bound of code length given by [30, Theorem 40] is 251, which differs from the actual code length of RM codes by only 5. Then $\mathcal{RM}(9,2)$ is also close to optimal, where the lower bound on code length is 500. However, for RM codes with larger order (dimension) and larger code length, the lower bound differs from the actual code length by at least 50, e.g., for $\mathcal{RM}(9,3)$, the lower bound becomes 464.

E. Optimal Scaling and Sharp Threshold of Reed-Muller Codes Over BSC Channels

Recently, Hassani et al. gave theoretical results backing the conjecture that RM codes have an almost optimal scaling-law over BSC channels under ML decoding [23], where optimal scaling-law means that for a fixed linear code, the decoding error probability of ML decoder transitions from 0 to 1 as a function of the crossover probability of the BSC channel in the sharpest manner (i.e., comparable to random codes). In particular, this implies that RM codes have sharper transition than

polar codes under ML decoding (if capacity achieving). In this section we give simulation results that show that for BSC channels, Reed-Muller codes under the RPA decoder also have sharper transition than polar codes under SCL+list decoder.

In Figure 6, we plot the decoding error probability of RM codes and polar codes over BSC channels as a function of the channel crossover probability, where for RM codes we use the RPA decoder in Algorithm 1, and for polar codes we use SCL decoder with list size 32. We can see that in all 4 cases, the transition in the curve of RM codes is sharper than the transition in the curve of polar codes. To further quantify the transition width, we introduce the following common notation: Let us denote the channel crossover probability as ϵ . For a given code and a corresponding decoding algorithm, we write its decoding error probability over BSC(ϵ) as $P_e(\epsilon)$. For $0 < \delta < 1/2$, we define the transition width⁹

$$w(\delta) := P_e^{-1}(1 - \delta) - P_e^{-1}(\delta).$$

Clearly, $w(\delta)$ is a decreasing function. For a fixed value of δ , smaller $w(\delta)$ means sharper transition and better scaling of the code and the corresponding decoder.

In Figure 7, we compare $w(0.1)$ and $w(0.01)$ between RM codes and polar codes with the same parameters, where we use the same decoders as above. We can see that RM codes always have smaller transition width than polar codes. Moreover, within the same code family, the transition width $w(0.1)$ and $w(0.01)$ both decrease with the code length, meaning that the transition becomes sharper as the code length increases. This phenomena has already been proved for ML decoders in [31] and [23].

VII. EXTENSIONS

Here we mention a few possible extensions of the decoding algorithms.

1. The “voting sets” idea to further accelerate the RPA decoding, as employed in Section V and discussed in Section VI-C.

2. Our new algorithms make use of one-dimensional subspace reduction. In practice, we can change the $\mathbb{B}_1, \dots, \mathbb{B}_{n-1}$ in the RPA decoding algorithms to any of the s -dimensional subspaces, with different combinations possible. Note that in Section V, we already made use of this idea, where we chose $s = 2$.

3. The RPA decoding algorithms can also be used to decode other codes that are supported on a vector space, or any code that has a well-defined notion of “code projection” that can be iteratively applied to produce eventually a trivial code (that can be decoded efficiently). In the case of RM codes, the quotient space projection has the specificity of producing again RM codes, and the trivial code is the Hadamard code that can be decoded using the FHT.

4. As discussed in Section III-A, we can use spectral decompositions or other relaxations in the Aggregation step instead of the majority voting, and depending on the regimes, one may take multiple iteration of the power-iteration method.

⁹Typically $P_e(\epsilon)$ is an increasing function of ϵ , so the inverse function exists.

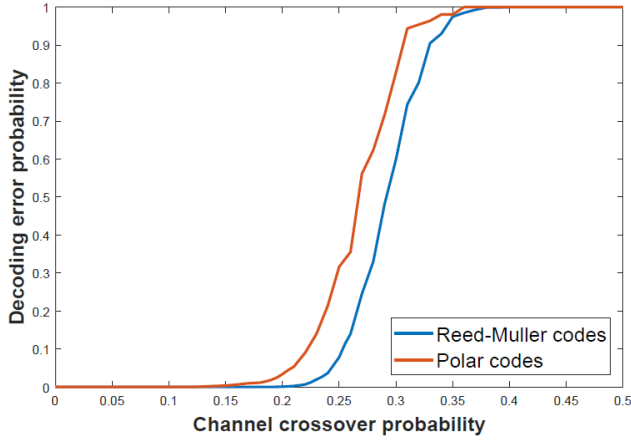
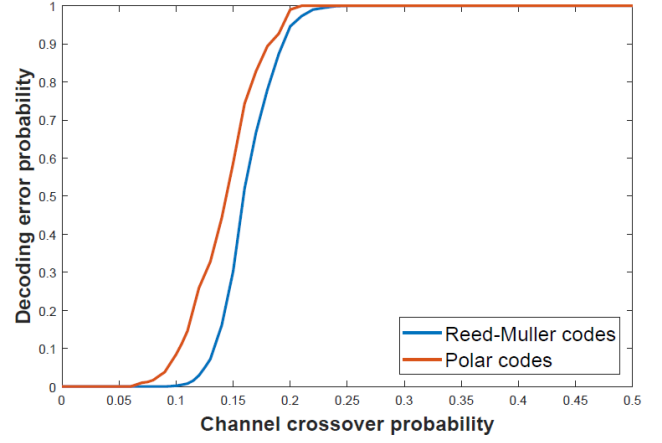
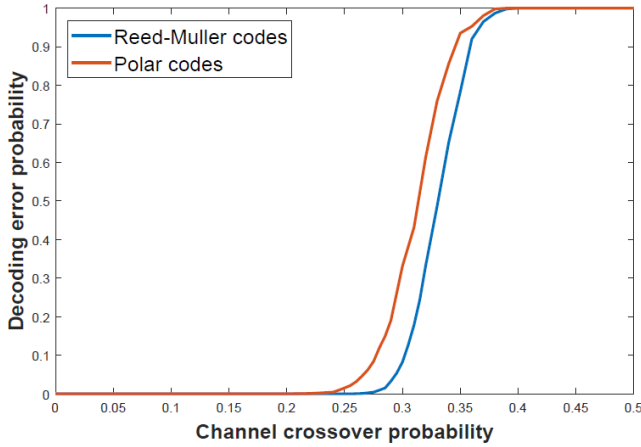
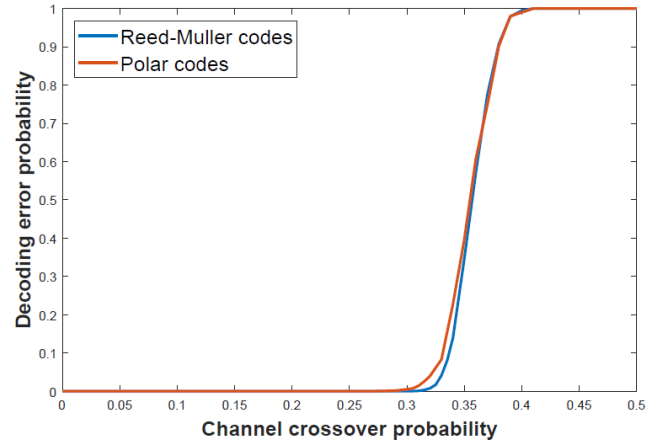
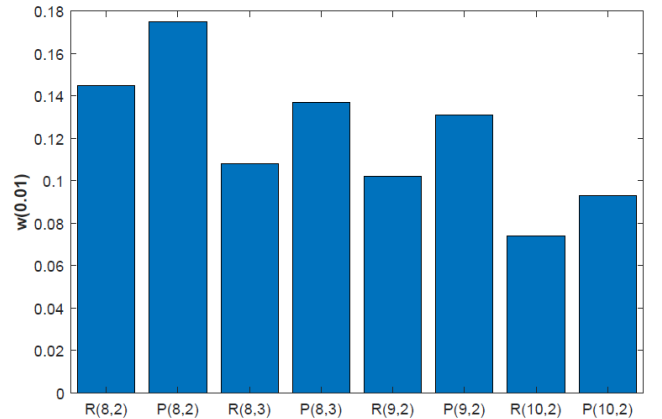
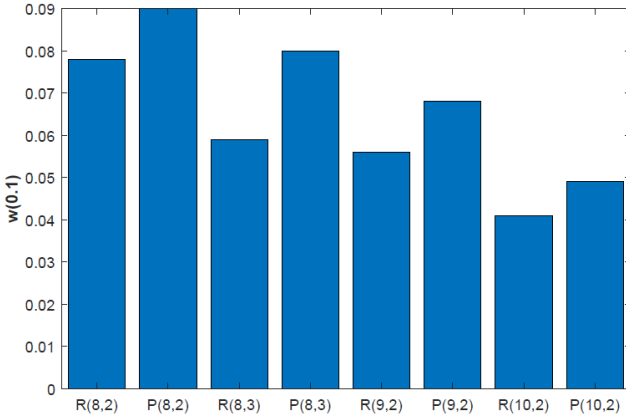
(a) $\mathcal{RM}(8, 2)$ vs Polar codes with the same parameters(b) $\mathcal{RM}(8, 3)$ vs Polar codes with the same parameters(c) $\mathcal{RM}(9, 2)$ vs Polar codes with the same parameters(d) $\mathcal{RM}(10, 2)$ vs Polar codes with the same parameters

Fig. 6. Decoding error probability over BSC channels as a function of the channel crossover probability.

Fig. 7. Comparison of transition width $w(0.1)$ and $w(0.01)$ between different codes. $R(m, r)$ refers to Reed-Muller codes, and $P(m, r)$ refers to polar codes with the same length and dimension as $R(m, r)$.

APPENDIX A PROOF OF LEMMA 1

Let $\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_m$ be a basis of $\mathbb{E}$ over $\mathbb{F}_2$ such that the first s vectors $\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_s$ form a basis of $\mathbb{B}$. Let $\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_m$ be the standard basis of $\mathbb{E}$, i.e., all but the i -th coordinate of

$\mathbf{e}_i$ are 0. Then there is an $m \times m$ invertible matrix M such that

$$(\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_m)^T = M(\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_m)^T.$$

Let $(z_1, z_2, \dots, z_m)$ be the coordinates of a point in $\mathbb{E}$ under the standard basis $(\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_m)$, and let $(z'_1, z'_2, \dots, z'_m)$

be the coordinates of the same point under the basis $(\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_m)$. Then

$$(z'_1, z'_2, \dots, z'_m) = (z_1, z_2, \dots, z_m)M^{-1}.$$

Notice that $\mathbb{B} = \{z : (z'_1, z'_2, \dots, z'_s) \in \mathbb{F}_2^s, z'_{s+1} = z'_{s+2} = \dots = z'_m = 0\}$. Therefore for every coset $T \in \mathbb{E}/\mathbb{B}$, the last $m - s$ coordinates under the basis $(\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_m)$ are the same for all the points in T . As a result, we can use binary vectors of length $m - s$ to label the cosets, i.e.,

$$\begin{aligned} [a_1, a_2, \dots, a_{m-s}] &:= \\ \{z : (z'_1, z'_2, \dots, z'_s) \in \mathbb{F}_2^s, \\ z'_{s+1} = a_1, z'_{s+2} = a_2, \dots, z'_m = a_{m-s}\}. \end{aligned}$$

Next we associate every subset $A \subseteq [m]$ with another row vector $\mathbf{v}'_m(A)$ of length 2^m , whose components are indexed by $\mathbf{z} = (z_1, z_2, \dots, z_m) \in \mathbb{E}$. The vector $\mathbf{v}'_m(A)$ is defined as follows:

$$\mathbf{v}'_m(A, \mathbf{z}) = \prod_{i \in A} z'_i,$$

where $\mathbf{v}'_m(A, \mathbf{z})$ is the component of $\mathbf{v}'_m(A)$ indexed by $\mathbf{z}$, i.e., $\mathbf{v}'_m(A, \mathbf{z})$ is the evaluation of the polynomial $\prod_{i \in A} Z'_i$ at $\mathbf{z}$, where $(Z'_1, Z'_2, \dots, Z'_m) = (Z_1, Z_2, \dots, Z_m)M^{-1}$. Since all the invertible linear transforms belong to the automorphism group of Reed-Muller codes [10], we have the following alternative characterization of RM codes

$$\begin{aligned} \mathcal{RM}(m, r) &:= \left\{ \sum_{A \subseteq [m], |A| \leq r} u'(A) \mathbf{v}'_m(A) : u'(A) \in \{0, 1\} \right. \\ &\quad \left. \text{for all } A \subseteq [m], |A| \leq r \right\}. \end{aligned}$$

It is easy to check that for every coset $T = [z'_{s+1}, z'_{s+2}, \dots, z'_m] \in \mathbb{E}/\mathbb{B}$, if $[s] \subseteq A$ then $\sum_{\mathbf{z} \in T} \mathbf{v}'_m(A, \mathbf{z}) = \prod_{i \in (A \setminus [s])} z'_i$, and if $[s] \not\subseteq A$ then $\sum_{\mathbf{z} \in T} \mathbf{v}'_m(A, \mathbf{z}) = 0$. Now let c be a codeword of $\mathcal{RM}(m, r)$, then it can be written as $c = \sum_{A \subseteq [m], |A| \leq r} u'(A) \mathbf{v}'_m(A)$, and for every coset $T = [z'_{s+1}, z'_{s+2}, \dots, z'_m] \in \mathbb{E}/\mathbb{B}$, we have

$$\begin{aligned} \sum_{\mathbf{z} \in T} c(\mathbf{z}) &= \sum_{A \supseteq [s], |A| \leq r} u'(A) \prod_{i \in (A \setminus [s])} z'_i \\ &= \sum_{A \subseteq ([m] \setminus [s]), |A| \leq r-s} u'(A) \prod_{i \in A} z'_i. \end{aligned}$$

Therefore every codeword in $\mathcal{Q}(m, r, \mathbb{B})$ corresponds to an $(m - s)$ -variate polynomial in $\mathbb{F}_2[Z'_{s+1}, Z'_{s+2}, \dots, Z'_m]$ with degree at most $r - s$, and this is exactly the definition of the $(r - s)$ -th order Reed-Muller code $\mathcal{RM}(m - s, r - s)$.

APPENDIX B PROOF OF PROPOSITION 4

We need the following technical lemma to prove Proposition 4.

Lemma 3: Let $c_0 = (c_0(\mathbf{z}), \mathbf{z} \in \mathbb{E})$ be a codeword of $\mathcal{RM}(m, r)$. Let $L^{(1)} = (L^{(1)}(\mathbf{z}), \mathbf{z} \in \mathbb{E})$ and $L^{(2)} = (L^{(2)}(\mathbf{z}), \mathbf{z} \in \mathbb{E})$ be two LLR vectors such that

$$L^{(2)}(\mathbf{z}) = (-1)^{c_0(\mathbf{z})} L^{(1)}(\mathbf{z}) \quad \forall \mathbf{z} \in \mathbb{E}. \quad (16)$$

Denote $\hat{c}_1 = \text{RPA_RM}(L^{(1)}, m, r, N_{\max}, \theta)$ and $\hat{c}_2 = \text{RPA_RM}(L^{(2)}, m, r, N_{\max}, \theta)$. Then $\hat{c}_1 = \hat{c}_2 + c_0$.

Proof: We prove by induction on r . For the base case $r = 1$, we use the ML decoder as described at the beginning of this section. More precisely, according to (9), $\hat{c}_2 = \text{RPA_RM}(L^{(2)}, m, 1, N_{\max}, \theta)$ is the codeword in $\mathcal{RM}(m, 1)$ that maximizes

$$\sum_{\mathbf{z} \in \mathbb{E}} \left((-1)^{c(\mathbf{z})} L^{(2)}(\mathbf{z}) \right),$$

i.e., for all $c \in \mathcal{RM}(m, 1)$, we have

$$\sum_{\mathbf{z} \in \mathbb{E}} \left((-1)^{\hat{c}_2(\mathbf{z})} L^{(2)}(\mathbf{z}) \right) \geq \sum_{\mathbf{z} \in \mathbb{E}} \left((-1)^{c(\mathbf{z})} L^{(2)}(\mathbf{z}) \right).$$

By (16), for all $c \in \mathcal{RM}(m, 1)$, we have

$$\sum_{\mathbf{z} \in \mathbb{E}} \left((-1)^{\hat{c}_2(\mathbf{z}) \oplus c_0(\mathbf{z})} L^{(1)}(\mathbf{z}) \right) \geq \sum_{\mathbf{z} \in \mathbb{E}} \left((-1)^{c(\mathbf{z}) \oplus c_0(\mathbf{z})} L^{(1)}(\mathbf{z}) \right).$$

Since c_0 is a codeword of $\mathcal{RM}(m, 1)$, we have: $c_0 + \mathcal{RM}(m, 1) = \mathcal{RM}(m, 1)$. As a result, for all $c \in \mathcal{RM}(m, 1)$, we have

$$\sum_{\mathbf{z} \in \mathbb{E}} \left((-1)^{\hat{c}_2(\mathbf{z}) \oplus c_0(\mathbf{z})} L^{(1)}(\mathbf{z}) \right) \geq \sum_{\mathbf{z} \in \mathbb{E}} \left((-1)^{c(\mathbf{z})} L^{(1)}(\mathbf{z}) \right).$$

Therefore, $\hat{c}_2 \oplus c_0$ is the codeword in $\mathcal{RM}(m, 1)$ that maximizes

$$\sum_{\mathbf{z} \in \mathbb{E}} \left((-1)^{c(\mathbf{z})} L^{(1)}(\mathbf{z}) \right).$$

Thus we conclude that $\hat{c}_1 = \hat{c}_2 \oplus c_0$. This establishes the base case.

For the inductive step, let us assume that the lemma holds for $r - 1$ and prove it for r . Notice that in Algorithm 3, $\hat{c}(\mathbf{z})$ is simply determined by the sign of $L(\mathbf{z})$. It is easy to see that if in Algorithm 4, the updated LLR vectors $\hat{L}^{(1)}$ and $\hat{L}^{(2)}$ always satisfy (16), then $\hat{c}_1 = \hat{c}_2 \oplus c_0$. Therefore, we only need to prove (16) for the updated LLR vectors $\hat{L}^{(1)}$ and $\hat{L}^{(2)}$.

Assuming that $L^{(1)}$ and $L^{(2)}$ satisfy (16), our task is to show that $\hat{L}^{(2)}(\mathbf{z}) = (-1)^{c_0(\mathbf{z})} \hat{L}^{(1)}(\mathbf{z})$ for all $\mathbf{z} \in \mathbb{E}$. From the analysis in Section IV, we know that

$$\hat{L}^{(i)}(\mathbf{z}) = \frac{1}{n-1} \sum_{\mathbf{z}' \neq \mathbf{z}} \alpha_i(\mathbf{z}, \mathbf{z}') L^{(i)}(\mathbf{z}') \quad \text{for } i = 1, 2. \quad (17)$$

The coefficient $\alpha_i(\mathbf{z}, \mathbf{z}')$ is 1 if the decoding result of the corresponding $(r - 1)$ th order RM code at the coset $\{\mathbf{z}, \mathbf{z}'\}$ is 0, and $\alpha_i(\mathbf{z}, \mathbf{z}')$ is -1 if the decoding result at the coset $\{\mathbf{z}, \mathbf{z}'\}$ is 1 (see line 3 of Algorithm 4).

Next we will show that $\alpha_2(\mathbf{z}, \mathbf{z}') = (-1)^{c_0(\mathbf{z}) \oplus c_0(\mathbf{z}')} \alpha_1(\mathbf{z}, \mathbf{z}')$. Note that $\alpha_i(\mathbf{z}, \mathbf{z}')$ is determined by the decoding result $\hat{y}_{\mathbb{B}}^{(i)} = \text{RPA_RM}(L^{(i)}_{/\mathbb{B}}, m - 1, r - 1, N_{\max}, \theta)$, where $\mathbb{B} = \{0, \mathbf{z} \oplus \mathbf{z}'\}$.

By (13), we have

$$\begin{aligned}
L_{/\mathbb{B}}^{(2)}(T) &= \ln \left(\exp \left(\sum_{z \in T} L^{(2)}(z) \right) + 1 \right) - \ln \left(\sum_{z \in T} \exp(L^{(2)}(z)) \right) \\
&= \ln \left(\exp \left(\sum_{z \in T} (-1)^{c_0(z)} L^{(1)}(z) \right) + 1 \right) \\
&\quad - \ln \left(\sum_{z \in T} \exp((-1)^{c_0(z)} L^{(1)}(z)) \right) \\
&= (-1)^{\bigoplus_{z \in T} c_0(z)} \left(\ln \left(\exp \left(\sum_{z \in T} L^{(1)}(z) \right) + 1 \right) \right. \\
&\quad \left. - \ln \left(\sum_{z \in T} \exp(L^{(1)}(z)) \right) \right) \\
&= (-1)^{\bigoplus_{z \in T} c_0(z)} L_{/\mathbb{B}}^{(1)}(T).
\end{aligned}$$

Let us write $c_0(T) := \bigoplus_{z \in T} c_0(z)$. Then $L_{/\mathbb{B}}^{(2)}(T) = (-1)^{c_0(T)} L_{/\mathbb{B}}^{(1)}(T)$ for all $T \in \mathbb{E}/\mathbb{B}$. Moreover, since c_0 is a codeword of $\mathcal{RM}(m, r)$ and $\mathbb{B}$ is a one-dimensional subspace of $\mathbb{E}$, by Lemma 1 we know that $(c_0(T), T \in \mathbb{E}/\mathbb{B})$ is a codeword of $\mathcal{RM}(m-1, r-1)$. Therefore, the codeword $(c_0(T), T \in \mathbb{E}/\mathbb{B})$ and the two LLR vectors $(L_{/\mathbb{B}}^{(1)}(T), T \in \mathbb{E}/\mathbb{B})$ and $(L_{/\mathbb{B}}^{(2)}(T), T \in \mathbb{E}/\mathbb{B})$ satisfy the conditions of this lemma. By the induction hypothesis, $\hat{y}_{/\mathbb{B}}^{(2)}(T) = \hat{y}_{/\mathbb{B}}^{(1)}(T) \oplus c_0(T)$ for all $T \in \mathbb{E}/\mathbb{B}$. As a result, we have $\alpha_2(z, z') = (-1)^{c_0(z) \oplus c_0(z')} \alpha_1(z, z')$. Taking this into (17), we conclude that for all $z \in \mathbb{E}$,

$$\begin{aligned}
\hat{L}^{(2)}(z) &= \frac{1}{n-1} \sum_{z' \neq z} \alpha_2(z, z') L^{(2)}(z') \\
&= \frac{1}{n-1} \sum_{z' \neq z} \left((-1)^{c_0(z) \oplus c_0(z')} \alpha_1(z, z') (-1)^{c_0(z')} L^{(1)}(z') \right) \\
&= (-1)^{c_0(z)} \frac{1}{n-1} \sum_{z' \neq z} \alpha_1(z, z') L^{(1)}(z') = (-1)^{c_0(z)} \hat{L}^{(1)}(z).
\end{aligned}$$

This completes the proof of the inductive step and establishes the lemma. $\square$

Proof of Proposition 4: Since W is a BMS channel, there is a permutation π of the output alphabet $\mathcal{W}$ satisfying the two conditions in Definition 3. Since both c_1 and c_2 are codewords of $\mathcal{RM}(m, r)$, $c_0 := c_1 + c_2$ is also a codeword of $\mathcal{RM}(m, r)$. Clearly, both channel output vectors Y_1 and Y_2 belong to $\mathcal{W}^n$. Now we define a permutation π^{c_0} on $\mathcal{W}^n$: For any $y = (y(z), z \in \mathbb{E}) \in \mathcal{W}^n$,

$$\pi^{c_0}(y) := (\pi^{c_0(z)}(y(z)), z \in \mathbb{E}).$$

Notice that $c_0(z)$ is either 0 or 1, and π^0 is the identity map. Since π is a permutation on $\mathcal{W}$, π^{c_0} is clearly a permutation on $\mathcal{W}^n$. For a given $y = (y(z), z \in \mathbb{E}) \in \mathcal{W}^n$, we denote the LLR vector corresponding to y as $L_y^{(1)} := (L_y^{(1)}(z), z \in \mathbb{E})$,

Algorithm 8 The RPA_RM Decoding Function for BSC

Input: The corrupted codeword $y = (y(z), z \in \{0, 1\}^m)$; the parameters of the Reed-Muller code m and r ; the maximal number of iterations $N_{\max}$

Output: The decoded codeword $\hat{c}$

```

1: for  $i = 1, 2, \dots, N_{\max}$  do
2:   Initialize  $(\text{changevote}(z), z \in \{0, 1\}^m)$  as an all-zero
   vector indexed by  $z \in \{0, 1\}^m$ 
3:   for each non-zero  $z_0 \in \{0, 1\}^m$  do
4:     Set  $\mathbb{B} = \{0, z_0\}$ 
5:      $\hat{y}_{/\mathbb{B}} \leftarrow \text{RPA\_RM}(y_{/\mathbb{B}}, m-1, r-1, N_{\max})$ 
6:      $\triangleright$  If  $r = 2$ , then we use the Fast Hadamard
   Transform to decode the first-order RM code [10]
7:     for each  $z \in \{0, 1\}^m$  do
8:       if  $y_{/\mathbb{B}}([z + \mathbb{B}]) \neq \hat{y}_{/\mathbb{B}}([z + \mathbb{B}])$  then
9:          $\text{changevote}(z) \leftarrow \text{changevote}(z) + 1 \triangleright$ 
   Here addition is between real numbers
10:      end if
11:    end for
12:  end for
13:   $\text{numofchange} \leftarrow 0$ 
14:   $n \leftarrow 2^m$ 
15:  for each  $z \in \{0, 1\}^m$  do
16:    if  $\text{changevote}(z) > \frac{n-1}{2}$  then
17:       $y(z) \leftarrow y(z) \oplus 1 \triangleright$  Here addition is over  $\mathbb{F}_2$ 
18:       $\text{numofchange} \leftarrow \text{numofchange} + 1 \triangleright$  Here
   addition is between real numbers
19:    end if
20:  end for
21:  if  $\text{numofchange} = 0$  then
22:    break  $\triangleright$  Exit the first for loop of this function
23:  end if
24:  end for
25:   $\hat{c} \leftarrow y$ 
26: return  $\hat{c}$ 

```

i.e., $L_y^{(1)}(z) = \text{LLR}(y(z))$ for all $z \in \mathbb{E}$, and we denote the LLR vector corresponding to $\pi^{c_0}(y)$ as $L_y^{(2)} := (L_y^{(2)}(z), z \in \mathbb{E})$, i.e., $L_y^{(2)}(z) = \text{LLR}(\pi^{c_0(z)}(y(z)))$ for all $z \in \mathbb{E}$. By the property of π (see Definition 3), we have

$$L_y^{(2)}(z) = (-1)^{c_0(z)} L_y^{(1)}(z) \quad \forall z \in \mathbb{E}.$$

Since $c_0 \in \mathcal{RM}(m, r)$, by Lemma 3 we know that

$$\begin{aligned}
&\text{RPA_RM}(L_y^{(1)}, m, r, N_{\max}, \theta) \\
&= \text{RPA_RM}(L_y^{(2)}, m, r, N_{\max}, \theta) + c_0.
\end{aligned}$$

As a result, $\text{RPA_RM}(L_y^{(1)}, m, r, N_{\max}, \theta) \neq c_1$ if and only if $\text{RPA_RM}(L_y^{(2)}, m, r, N_{\max}, \theta) \neq c_2$.

For a vector $y \in \mathcal{W}^n$ and a codeword $c \in \mathcal{RM}(m, r)$, we use $W^n(y|c)$ to denote the probability of outputting y when the transmitted codeword is c . Again by the property of π , it is easy to see that

$$W^n(y|c_1) = W^n(\pi^{c_0}(y)|c_2) \quad \forall y \in \mathcal{W}^n.$$

Algorithm 9 The RPA_RM Decoding Function for General Binary-Input Memoryless Channels

Input: The LLR vector $(L(z), z \in \{0, 1\}^m)$; the parameters of the Reed-Muller code m and r ; the maximal number of iterations $N_{\max}$; the exiting threshold θ

Output: The decoded codeword $\hat{c} = (\hat{c}(z), z \in \{0, 1\}^m)$

```

1:  $\mathbb{E} := \{0, 1\}^m$ 
2: for  $i = 1, 2, \dots, N_{\max}$  do
3:   Initialize  $(\text{cumuLLR}(z), z \in \mathbb{E})$  as an all-zero vector indexed by  $z \in \mathbb{E}$ 
4:   for each non-zero  $z_0 \in \mathbb{E}$  do
5:     Set  $\mathbb{B} = \{0, z_0\}$ 
6:      $L_{/\mathbb{B}} \leftarrow (L_{/\mathbb{B}}(T), T \in \mathbb{E}/\mathbb{B}) \triangleright L_{/\mathbb{B}}(T)$  is calculated from  $(L(z), z \in \mathbb{E})$  according to (13)
7:      $\hat{y}_{/\mathbb{B}} \leftarrow \text{RPA\_RM}(L_{/\mathbb{B}}, m-1, r-1, N_{\max}, \theta)$ 
8:      $\triangleright$  If  $r = 2$ , then we use the Fast Hadamard Transform to decode the first-order RM code
9:     for each  $z \in \mathbb{E}$  do
10:      if  $\hat{y}_{/\mathbb{B}}([z + \mathbb{B}]) = 0$  then
11:         $\text{cumuLLR}(z) \leftarrow \text{cumuLLR}(z) + L(z \oplus z_0)$ 
12:      else  $\triangleright \hat{y}_{/\mathbb{B}}$  is the decoded codeword, so  $\hat{y}_{/\mathbb{B}}([z + \mathbb{B}])$  is either 0 or 1
13:         $\text{cumuLLR}(z) \leftarrow \text{cumuLLR}(z) - L(z \oplus z_0)$ 
14:      end if
15:    end for
16:  end for
17:   $\text{numofchange} \leftarrow 0$ 
18:   $n \leftarrow 2^m$ 
19:  for each  $z \in \mathbb{E}$  do
20:     $\text{cumuLLR}(z) \leftarrow \frac{\text{cumuLLR}(z)}{n-1}$ 
21:    if  $|\text{cumuLLR}(z) - L(z)| > \theta |L(z)|$  then
22:       $\text{numofchange} \leftarrow \text{numofchange} + 1$ 
23:       $\triangleright$  Here addition is between real numbers
24:    end if
25:     $L(z) \leftarrow \text{cumuLLR}(z)$ 
26:  end for
27:  if  $\text{numofchange} = 0$  then
28:    break  $\triangleright$  Exit the first for loop of this function
29:  end if
30: end for
31: for each  $z \in \mathbb{E}$  do
32:  if  $L(z) > 0$  then
33:     $\hat{c}(z) \leftarrow 0$ 
34:  else
35:     $\hat{c}(z) \leftarrow 1$ 
36:  end if
37: end for
38: return  $\hat{c}$ 

```

Recall that in Proposition 4, we use $L^{(1)}$ and $L^{(2)}$ to denote the random LLR vectors corresponding to the random channel outputs when transmitting c_1 and c_2 , respectively. Therefore,

$$\begin{aligned} & \mathbb{P}(\text{RPA_RM}(L^{(1)}, m, r, N_{\max}, \theta) \neq c_1) \\ &= \sum_{y \in \mathcal{W}^n} W^n(y|c_1) \mathbb{1}[\text{RPA_RM}(L_y^{(1)}, m, r, N_{\max}, \theta) \neq c_1] \end{aligned}$$

$$\begin{aligned} &= \sum_{y \in \mathcal{W}^n} W^n(\pi^{c_0}(y)|c_2) \mathbb{1}[\text{RPA_RM}(L_y^{(2)}, m, r, N_{\max}, \theta) \neq c_2] \\ &= \mathbb{P}(\text{RPA_RM}(L^{(2)}, m, r, N_{\max}, \theta) \neq c_2). \end{aligned}$$

This completes the proof of Proposition 4. $\square$

APPENDIX C

ANOTHER VERSION OF ALGORITHMS 1–4

See Algorithms 8 and 9.

ACKNOWLEDGMENT

The authors would like to thank Alexander Barg and Ilya Dumer for pointing out several references and giving useful feedback. They also thank Kirill Ivanov for useful discussions and feedback.

REFERENCES

- [1] M. Ye and E. Abbe, "Recursive projection-aggregation decoding of Reed-Muller codes," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jul. 2019, pp. 2064–2068.
- [2] I. Reed, "A class of multiple-error-correcting codes and the decoding scheme," *Trans. IRE Prof. Group Inf. Theory*, vol. 4, no. 4, pp. 38–49, Sep. 1954.
- [3] E. Arikan, "Channel polarization: A method for constructing capacity-achieving codes for symmetric binary-input memoryless channels," *IEEE Trans. Inf. Theory*, vol. 55, no. 7, pp. 3051–3073, Jul. 2009.
- [4] E. Arkan, "A performance comparison of polar codes and Reed-Muller codes," *IEEE Commun. Lett.*, vol. 12, no. 6, pp. 447–449, Jun. 2008.
- [5] M. Mondelli, S. H. Hassani, and R. L. Urbanke, "From polar to Reed-Muller codes: A technique to improve the finite-length performance," *IEEE Trans. Commun.*, vol. 62, no. 9, pp. 3084–3091, Sep. 2014.
- [6] S. Kudekar, S. Kumar, M. Mondelli, H. D. Pfister, E. Şaşıoğlu, and R. L. Urbanke, "Reed-Muller codes achieve capacity on erasure channels," *IEEE Trans. Inf. Theory*, vol. 63, no. 7, pp. 4298–4316, Jul. 2017.
- [7] E. Abbe, A. Shpilka, and A. Wigderson, "Reed-Muller codes for random erasures and errors," *IEEE Trans. Inf. Theory*, vol. 61, no. 10, pp. 5229–5252, Oct. 2015.
- [8] E. Abbe and M. Ye, "Reed-Muller codes polarize," in *Proc. IEEE 60th Annu. Symp. Found. Comput. Sci. (FOCS)*, Nov. 2019, pp. 273–286.
- [9] E. Abbe, A. Shpilka, and M. Ye, "Reed-Muller codes: Theory and algorithms," 2020, *arXiv:2002.03317*. [Online]. Available: <http://arxiv.org/abs/2002.03317>
- [10] F. J. MacWilliams and N. J. A. Sloane, *The Theory of Error-Correcting Codes*. Amsterdam, The Netherlands: Elsevier, 1977.
- [11] V. M. Sidel'nikov and A. S. Pershakov, "Decoding of Reed-Muller codes with a large number of errors," *Probl. Peredachi Inf.*, vol. 28, no. 3, pp. 80–94, 1992.
- [12] P. Loidreau and B. Sakkour, "Modified version of Sidel'nikov-Pershakov decoding algorithm for binary second order Reed-Muller codes," in *Proc. 9th Int. Workshop Algebraic Combinat. Coding Theory (ACCT)*, Kranevo, Bulgaria, 2004, pp. 266–271.
- [13] B. Sakkour, "Decoding of second order Reed-Muller codes with a large number of errors," in *Proc. IEEE Inf. Theory Workshop*, Sep. 2005, pp. 176–178.
- [14] I. Dumer, "Recursive decoding and its performance for low-rate Reed-Muller codes," *IEEE Trans. Inf. Theory*, vol. 50, no. 5, pp. 811–823, May 2004.
- [15] I. Dumer, "Soft-decision decoding of Reed-Muller codes: A simplified algorithm," *IEEE Trans. Inf. Theory*, vol. 52, no. 3, pp. 954–963, Mar. 2006.
- [16] I. Dumer and K. Shabunov, "Soft-decision decoding of Reed-Muller codes: Recursive lists," *IEEE Trans. Inf. Theory*, vol. 52, no. 3, pp. 1260–1266, Mar. 2006.
- [17] R. Satharishi, A. Shpilka, and B. L. Volk, "Efficiently decoding Reed-Muller codes from random errors," *IEEE Trans. Inf. Theory*, vol. 63, no. 4, pp. 1954–1960, Apr. 2016.
- [18] O. Sberlo and A. Shpilka, "On the performance of Reed-Muller codes with respect to random errors and erasures," *Proc. 14th Annu. ACM-SIAM Symp. Discrete Algorithms*, Dec. 2020, pp. 1357–1376.

- [19] E. Santi, C. Hager, and H. D. Pfister, "Decoding Reed–Muller codes using minimum-weight parity checks," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jun. 2018, pp. 1296–1300.
- [20] I. Tal and A. Vardy, "List decoding of polar codes," *IEEE Trans. Inf. Theory*, vol. 61, no. 5, pp. 2213–2226, May 2015.
- [21] D. Chase, "Class of algorithms for decoding block codes with channel measurement information," *IEEE Trans. Inf. Theory*, vol. IT-18, no. 1, pp. 170–182, Jan. 1972.
- [22] *Final Report of 3 GPP TSG RAN WG1 #87 v1.0.0*. Accessed: Feb. 8, 2017. [Online]. Available: http://www.3gpp.org/ftp/tsg_ran/WG1_RL1/TSGR1_87/Report/
- [23] H. Hassani, S. Kudekar, O. Ordentlich, Y. Polyanskiy, and R. Urbanke, "Almost optimal scaling of Reed–Muller codes on BEC and BSC channels," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jun. 2018, pp. 311–315.
- [24] R. R. Green, "A serial orthogonal decoder," *JPL Space Programs Summary*, vol. IV, nos. 37–39, pp. 247–253, 1966.
- [25] Y. Be'ery and J. Snyders, "Optimal soft decision block decoders based on fast Hadamard transform," *IEEE Trans. Inf. Theory*, vol. IT-32, no. 3, pp. 355–364, May 1986.
- [26] A. Balatsoukas-Stimming, M. Bastani Parizi, and A. Burg, "LLR-based successive cancellation list decoding of polar codes," *IEEE Trans. Signal Process.*, vol. 63, no. 19, pp. 5165–5179, Oct. 2015.
- [27] G. Sarkis, P. Giard, A. Vardy, C. Thibault, and W. J. Gross, "Fast list decoders for polar codes," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 2, pp. 318–328, Feb. 2016.
- [28] S. A. Hashemi, N. Doan, M. Mondelli, and W. J. Gross, "Decoding Reed–Muller and polar codes by successive factor graph permutations," in *Proc. IEEE 10th Int. Symp. Turbo Codes Iterative Inf. Process. (ISTC)*, Dec. 2018, pp. 1–5.
- [29] Y. Polyanskiy, H. V. Poor, and S. Verdú, "Channel coding rate in the finite blocklength regime," *IEEE Trans. Inf. Theory*, vol. 56, no. 5, pp. 2307–2359, May 2010.
- [30] Y. Polyanskiy, *Channel Coding: Non-Asymptotic Fundamental Limits*. Princeton, NJ, USA: Princeton Univ., 2010.
- [31] J.-P. Tillich and G. Zémor, "Discrete isoperimetric inequalities and the probability of a decoding error," *Combinat. Probab. Comput.*, vol. 9, no. 5, pp. 465–479, Sep. 2000.

Min Ye received the B.S. degree in electrical engineering from Peking University, Beijing, China, in 2012, and the Ph.D. degree from the Department of Electrical and Computer Engineering, University of Maryland, College Park, MA, USA, in 2017. He then spent two years as a Postdoctoral Researcher at Princeton University. Since 2019, he has been an Assistant Professor with the Data Science and Information Technology Research Center, Tsinghua-Berkeley Shenzhen Institute, Shenzhen, China. His research interests include coding theory, information theory, differential privacy, and machine learning. He received the 2017 IEEE Data Storage Best Paper Award.

Emmanuel Abbe received the Ph.D. degree from the Department of Electrical Engineering and Computer Science, Massachusetts Institute of Technology, in 2008, and the M.S. degree from the Department of Mathematics, Ecole Polytechnique Fédérale de Lausanne (EPFL), Switzerland, in 2003. In 2012, he joined Princeton University as an Assistant Professor and became an Associate Professor in 2016, jointly in the Program for Applied and Computational Mathematics and the Department of Electrical Engineering. He has been an Associate Faculty with the Department of Mathematics, Princeton University, since 2016. He is also a Professor with the Mathematics Institute, School of Computer and Communication Sciences, EPFL. He is a recipient of the Foundation Latsis International Prize, the Bell Labs Prize, the NSF CAREER Award, the Google Faculty Research Award, the Walter Curtis Johnson Prize, and the von Neumann Fellowship from the Institute for Advanced Study.